

Module 5 Memory Management

Module:5	Memory Management	7 hours
Main memory management, Memory allocation strategies, Virtual memory: Hardware support for virtual memory (caching, TLB) – Paging - Segmentation - Demand Paging - Page Faults - Page Replacement -Thrashing - Working Set.		

Memory Management Unit

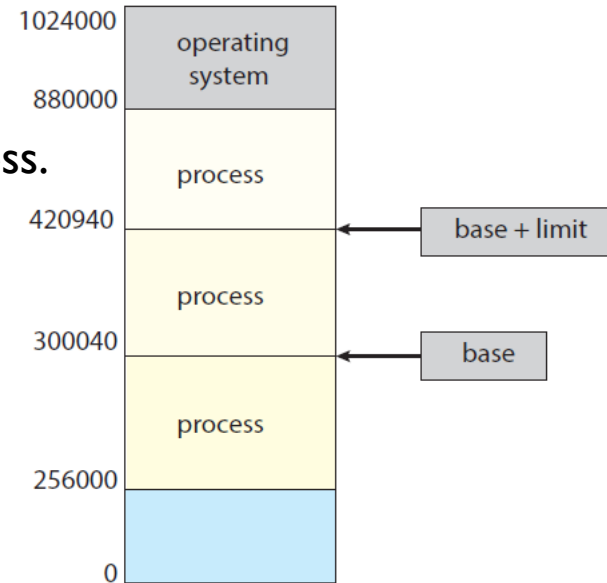
- Memory consists of a large array of bytes, each with its own address.
- The CPU fetches instructions from memory according to the value of the program counter.
- Main memory and the registers built into each processing core are the only general-purpose storage that the CPU can access directly.
- CPU can directly access only:
 - CPU registers
 - Main memory (RAM)
- The CPU cannot directly access data stored on disk.
 - Machine instructions can refer to memory addresses, but not disk addresses.
 - Therefore, programs and data must first be loaded into main memory before execution.
- Registers are the fastest storage locations.
 - They are built inside each CPU core.
 - They can usually be accessed within one CPU clock cycle.
- Main memory is slower than CPU registers.
 - Main memory is accessed through the memory bus.
 - A memory access may require several CPU clock cycles.

Memory Management Unit

- When the CPU is waiting for data from main memory, it may experience a **memory stall**.
 - The processor temporarily stops executing the current instruction until the required data arrives.
- Since memory accesses occur very frequently, memory stalls can significantly reduce CPU performance.
- To reduce this delay, a **cache memory** is placed between the CPU and main memory.
 - Cache is much faster than main memory.
 - It is generally located **inside or very close to the CPU**.
 - Frequently used instructions and data are stored in the cache.
- **Cache management is mainly handled automatically by hardware.**
 - The operating system normally does not control every cache access.
 - This helps improve memory-access performance.
- In a **multithreaded processor**, when one hardware thread is waiting for memory, the CPU may execute another hardware thread.

Memory Management Unit

- Each process must have its **own separate memory space**.
 - This prevents one process from accessing or modifying another process's memory.
 - It allows multiple processes to remain in memory and execute concurrently.
- The system must determine the **legal range of memory addresses** that each process can access.
- This protection can be implemented using two hardware registers:
 - **Base register**
 - **Limit register**
- The **base register** stores the **smallest legal physical memory address** of the process.
- The **limit register** stores the **size of the process's allowed memory range**.
- Example:
 - Base = **300040**
 - Limit = **120900**
 - Legal address range:
 - Starting address = **300040**
 - Ending address = **$300040 + 120900 - 1 = 420939$**
 - Therefore, the process can access addresses from **300040 to 420939**.



Memory Management Unit

Simple Illustration

Suppose: **Base = 300040 ; Limit = 120900**

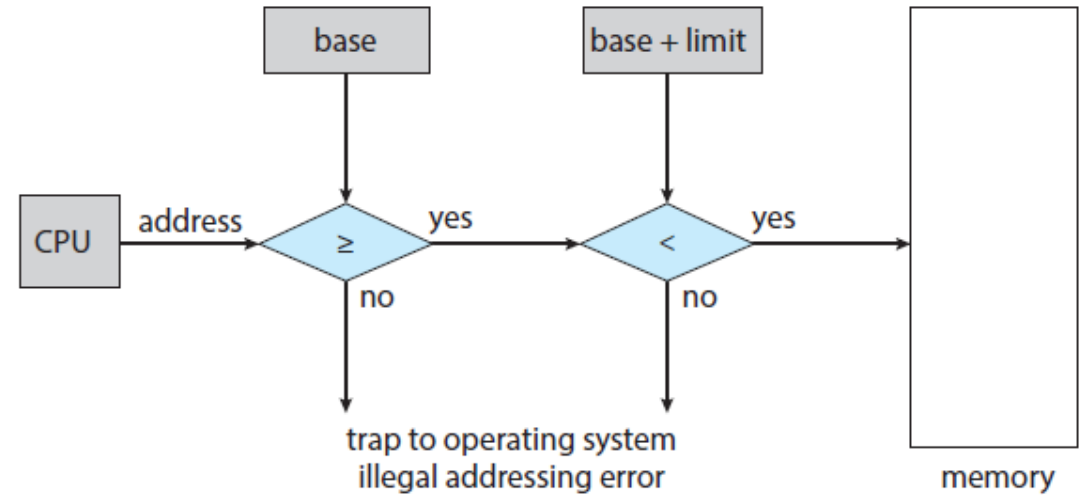
Then: **Valid memory range = 300040 to 420939**

If the CPU generates:

- **350000** → **Valid** → **Memory access allowed**
- **420000** → **Valid** → **Memory access allowed**
- **250000** → **Invalid** → **Trap to OS**
- **430000** → **Invalid** → **Trap to OS**

So, base and limit registers provide hardware-level memory protection by restricting each process to its assigned memory region.

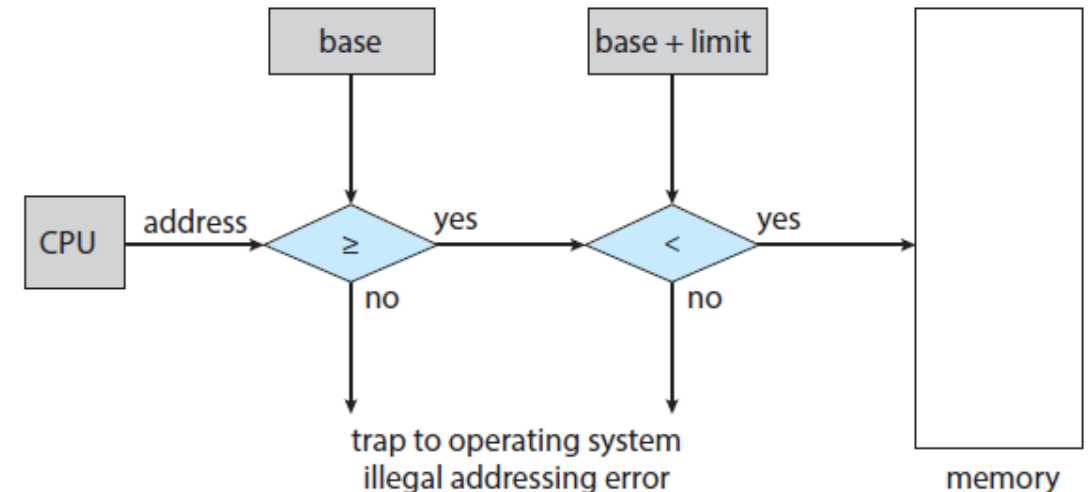
Hardware address protection with base and limit registers



Memory Management Unit

Memory Protection Using Base and Limit Registers:

- For every memory address generated by a program in **user mode**, the CPU hardware checks whether the address is within the process's legal memory range.
- The CPU compares the generated address with the **base and limit register values**.
- If the address is valid:
 - Memory access is permitted.
- If the address is outside the allowed range:
 - The CPU generates a **trap** to the operating system.
 - The operating system treats it as an **illegal or fatal memory-access error**.
- This mechanism prevents a user process from:
 - Accessing **operating-system memory**
 - Accessing **another process's memory**
 - Accidentally modifying protected memory
 - Deliberately modifying protected memory



Memory Management Unit

Who Can Modify the Base and Limit Registers?

- The **base and limit registers** can be loaded only by the operating system.
- Loading these registers requires a **privileged instruction**.
- Privileged instructions can execute only in **kernel mode**.
- User programs execute in **user mode**, so they cannot modify the base or limit register values.
- Therefore:
 - **OS can change the registers**
 - **User processes cannot change them**
- This prevents a process from increasing its own allowed memory range.

Memory Management Unit

Kernel Mode Memory Access

- The operating system runs in **kernel mode**.
- In kernel mode, the OS has **unrestricted access** to:
 - ☐ Operating-system memory
 - ☐ Memory belonging to user processes
- This unrestricted access is necessary because the OS must perform several system-management operations, such as:
 - ☐ Loading a program into memory
 - ☐ Reading or modifying user-process memory
 - ☐ Handling errors and program crashes
 - ☐ Accessing system-call parameters
 - ☐ Performing input/output operations involving user memory
 - ☐ Managing processes

Memory Management Unit

Example: During a **context switch**, the operating system must access the memory of different processes.

Suppose CPU is switching from **Process P1** to **Process P2**:

Step 1: P1 is currently running.

Step 2: OS enters **kernel mode**.

Step 3: OS saves P1's CPU state, such as:

- Program counter
- CPU registers
- Stack pointer
- into P1's memory or process control block.

Step 4: OS loads P2's previously saved state from memory into the CPU registers.

Step 5: P2 starts/resumes execution.

Therefore, the operating system requires unrestricted memory access during operations such as a **context switch**.

Address Binding

Program Loading and Address Binding:

- A program normally resides on **secondary storage (disk)** as a binary executable file.
- Before execution, the program must be:
 - Brought into **main memory**
 - Associated with a **process**
 - Made ready for execution by the CPU
- During execution, the process accesses its:
 - Instructions
 - Data
from main memory.
- When the process terminates, its allocated memory is released and reused by other processes.

Program need not start at address 0

- A user process can usually be placed **anywhere in physical memory**.
- Even if physical memory begins at address 00000, a process may start at some other address such as 50000.
- The operating system decides where the process is placed.

Address Binding

- During the program's life cycle, addresses can appear in different forms.

1. Symbolic Address

- Used in the source program.

Example: `int count;`

- Here, `count` is a **symbolic address/name**.

2. Relocatable Address

- Generated by the compiler.

Example:

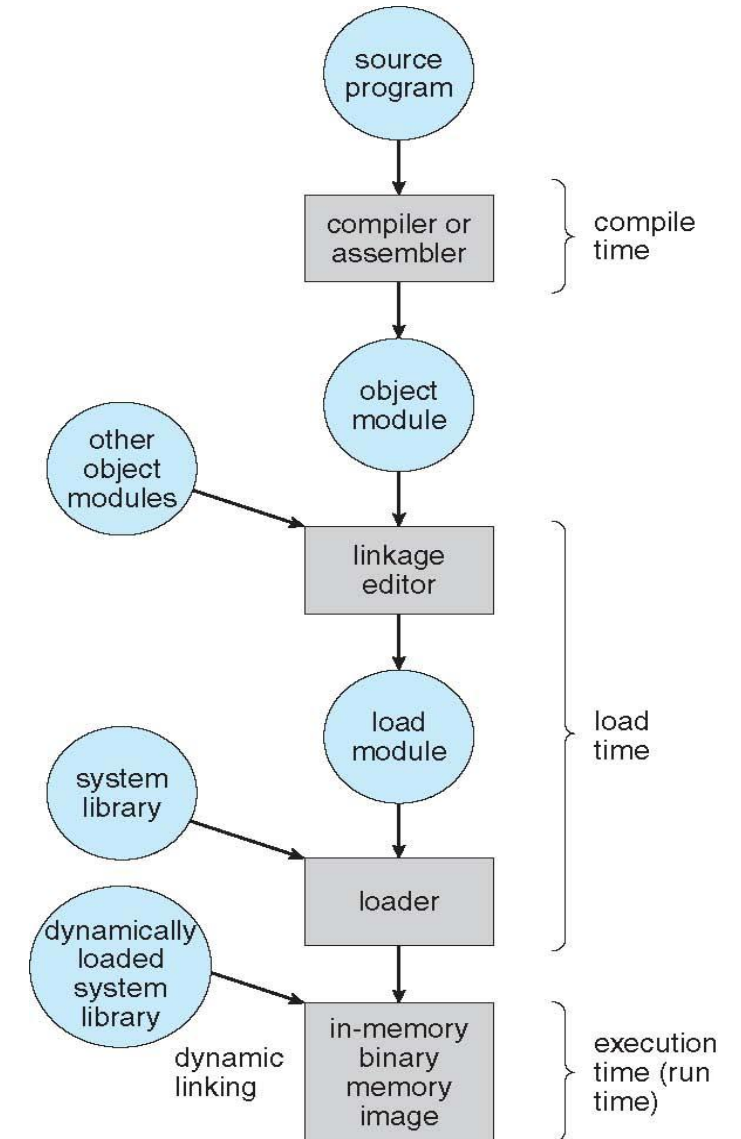
- 14 bytes from the beginning of the module
- It indicates an offset rather than a fixed physical location.

3. Absolute Address

- The actual memory address.
- Example: 74014

Thus:

Symbolic address → Relocatable address → Absolute address



Address Binding

Address binding means mapping a program's instructions and data to actual memory addresses.

Binding can occur at three different times:

I. Compile-Time Binding

- Used when the memory location of the program is known **during compilation**.
- Compiler generates **absolute code**.
- Example: Suppose the program is known to start at: 50000
- The compiler may directly generate addresses such as:

50000

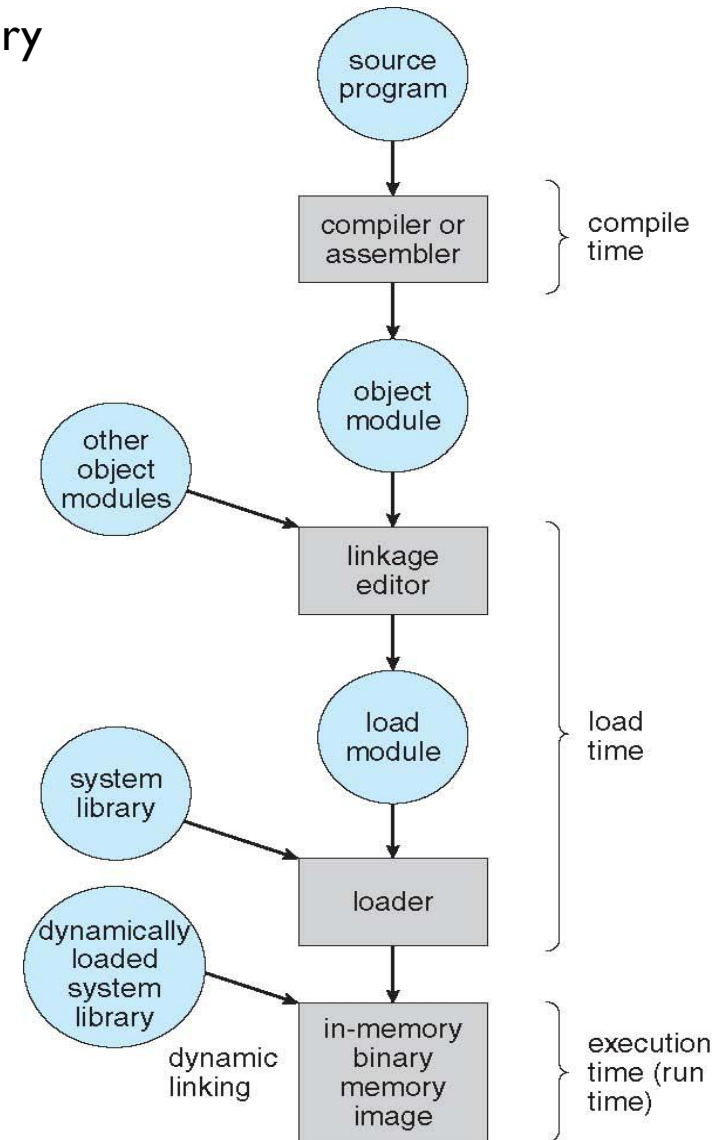
50004

50008

...

- If the starting memory location changes, the program must be **recompiled**.

Key point: Memory location known at compile time → **Absolute code**



Address Binding

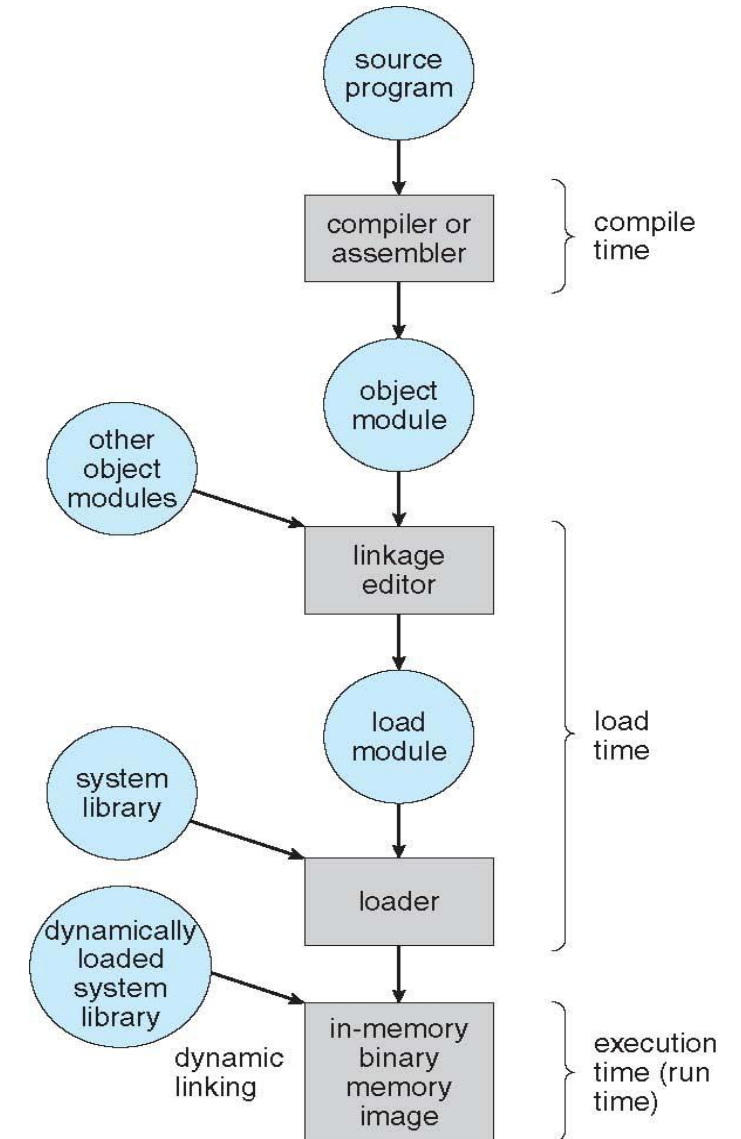
2. Load-Time Binding

- Used when the memory location is **not known during compilation**.
- Compiler generates **relocatable code**.
- Actual physical addresses are assigned when the program is loaded.

Example: Compiler generates: Instruction at offset 100

- If the loader places the program at: Base address = 50000, Then ,
Actual address = $50000 + 100$
 $= 50100$
- If the starting address changes, the program only needs to be **reloaded**, not recompiled.

Key point: Address decided while loading → **Relocatable code**



Address Binding

3. Execution-Time Binding

- Address binding is delayed until the program is actually running.
- The process can be moved from one area of memory to another **during execution**.
- Address translation is performed dynamically.

Example: Logical address generated by CPU = 500

If currently: Base = 30000

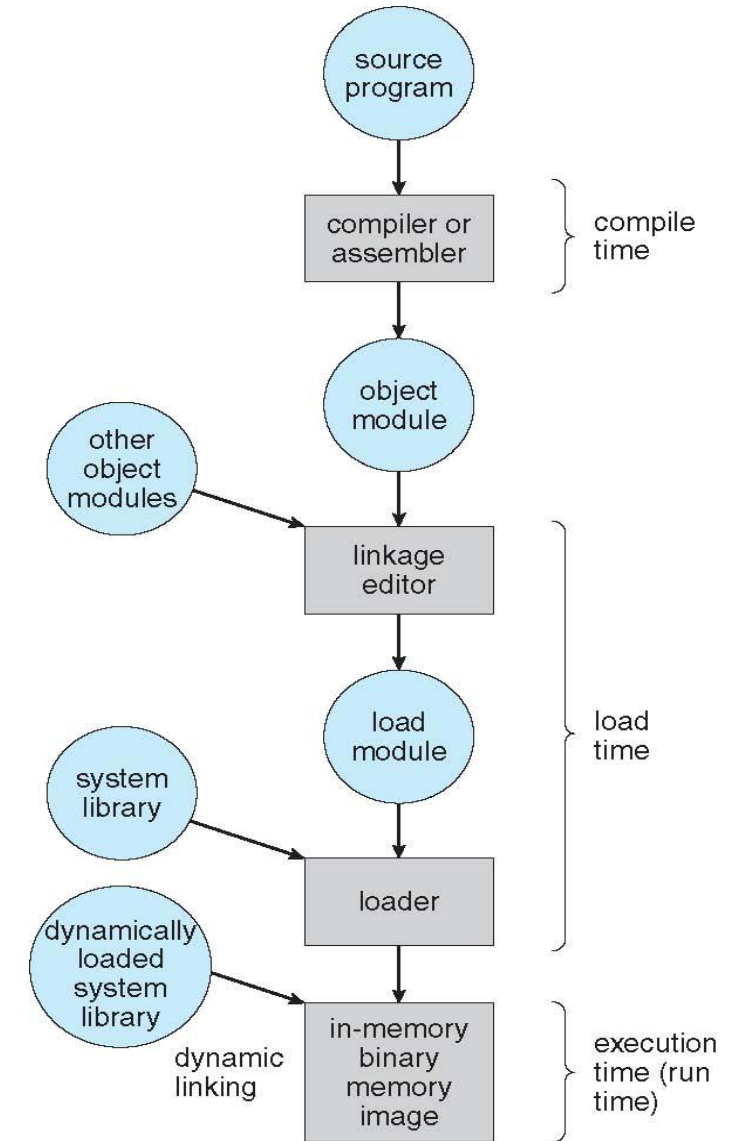
then: Physical address = 30500

Later, if the process is moved: Base = 60000

the same logical address becomes: Physical address = 60500

This requires special hardware support, particularly the

MMU (Memory Management Unit).



Logical vs. Physical Address Space

Logical Address

- A **logical address** is the address generated by the **CPU** while a program is executing. It is also called a **virtual address**.
- The program works with logical addresses and normally does not know the actual location in physical memory.
- Logical addresses are translated into physical addresses by the **MMU (Memory Management Unit)**.
- Example: Logical address generated by CPU = 500
- This does not necessarily mean the data is stored at physical memory location 500.

Physical Address

- A **physical address** is the actual address in **main memory (RAM)**.
- It is the address seen and used by the memory hardware.
- The MMU converts a logical address into the corresponding physical address.

Example: If Base / Relocation register = 14000; Logical address = 500

then: Physical address = 14000 + 500

= 14500

So: **CPU generates 500 → MMU converts it → RAM accesses 14500**

Logical vs. Physical Address Space

Logical Address Space

- The complete set of logical addresses generated by a process is called its **logical address space**.
- Each process has its own logical address space.

For **example**, a process may use: 0 to 9999 , even though it may actually be stored somewhere else in RAM.

Physical Address Space

- The complete set of physical addresses corresponding to a process is called its **physical address space**.
- It represents the actual locations occupied by that process in main memory.

Example: Suppose, Logical address space = 0 to 9999 , Base address = 30000

Then the corresponding physical address space is: 30000 to 39999

The MMU maps logical addresses to physical addresses during program execution.

Address translation

Suppose a program contains:

Logical address = 346 and the relocation register (also known as base register) contains: 14000

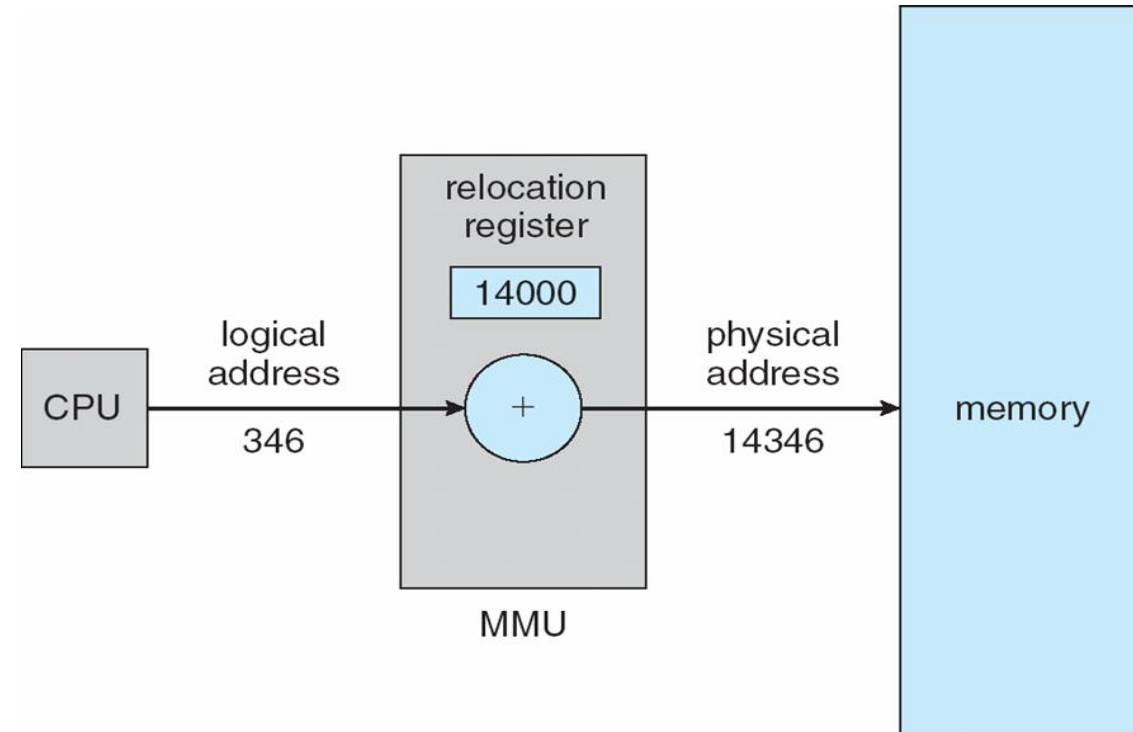
The MMU calculates:

$$\begin{aligned}\text{Physical address} &= \text{Relocation register} + \text{Logical address} \\ &= 14000 + 346 \\ &= 14346\end{aligned}$$

- So the program thinks it is accessing address **346**, while the actual RAM location is **14346**.

Summary

- Logical address = what the process sees.
- Physical address = where the data actually exists in RAM.
- The MMU maps logical addresses to physical addresses during program execution.



Dynamic Loading

Dynamic loading means a program module or routine is loaded into main memory **only when it is actually needed during execution**. That is **Load a part of the program only when required**.

- Instead of loading the entire program into memory at the beginning:
- Main program is loaded first.
- Other routines remain on disk.
- When a routine is called, the system checks whether it is already in memory. If not, that routine is loaded into memory and then executed.

Example: Suppose a program has:

- ✓ Main program
- ✓ Print module
- ✓ Error-handling module
- ✓ Report-generation module

If the user never generates a report, the report module is **never loaded into memory**.

Advantage

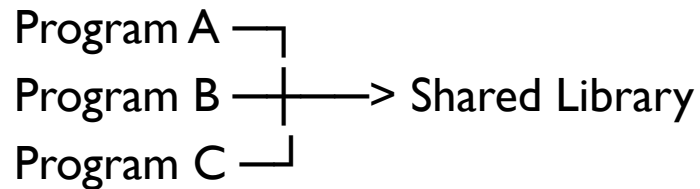
- Reduces memory usage.
- Useful for large programs.
- Rarely used routines do not occupy memory unnecessarily.

Dynamic Linking

Dynamic linking means linking a program to a required library during execution time rather than completely at compile/link time.

- The executable file contains a small piece of code called a **stub**.
- When the library function is called, the stub checks whether the required library is already available in memory.
- If necessary, the library is loaded or located.
- The program is then linked to that library function.

Example : Suppose multiple programs use the same library:



Instead of placing a separate copy of the library inside every executable, all programs can use a **shared library**.

Examples include: .dll files in Windows , .so files in Linux

Advantage

- ✓ Reduces executable file size.
- ✓ Saves memory.
- ✓ Multiple programs can share the same library.
- ✓ Library updates can sometimes be applied without recompiling the entire application.

Swapping

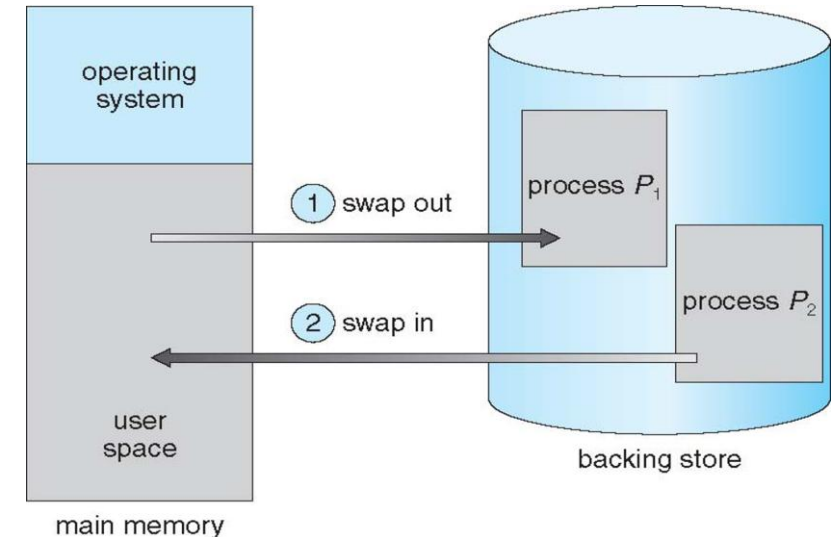
- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - Total physical memory space of processes can exceed physical memory
- **Backing store/swapping space** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images

Why swapping is needed

- Main memory is limited.
- Several processes may need to execute at the same time.
- If memory is insufficient, the operating system can temporarily remove an inactive or waiting process from RAM.
- This frees memory for another process.

The two main operations are:

- ✓ **Swap out** – move a process from RAM to disk.
- ✓ **Swap in** – bring the process back from disk to RAM.
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk



Swapping

Does the swapped out process need to swap back in to same physical addresses?

Not necessarily.

- If **execution-time address binding** is used, the process can be loaded into a different physical memory location when it is swapped back in.

For example:

Before swap: PI → Physical memory starting at 20000

After being swapped back: PI → Physical memory starting at 45000

- The **MMU** performs the necessary address translation.

Major disadvantage of swapping

- The main disadvantage is **high swapping time**.
- A large amount of data may need to be transferred between RAM and disk.

For example, if a process occupies **500 MB**, the OS may have to:

Copy 500 MB RAM → Disk during swap out

Copy 500 MB Disk → RAM during swap in

- This creates significant overhead. Therefore, frequent swapping can severely reduce system performance.

Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - Plus disk latency of 8 ms
 - Swap out time of 2008 ms
 - Plus swap in of same sized process
 - Total context switch swapping component time of 4016ms (> 4 seconds)

Consider:

- P1 is currently in RAM and executing.
- P2 is ready to execute but is currently on disk.
- There is not enough free RAM to bring P2 into memory.

The OS performs:

P1 in RAM → Save P1 CPU context → Swap P1 out to disk → Swap P2 from disk into RAM → Restore P2 CPU context → P2 starts executing

Total Switch Time = Context Save Time + Swap-out Time + Swap-in Time + Context Restore Time

Contiguous Allocation

- Contiguous allocation is a memory-management technique in which each process is loaded into **one continuous block of physical memory**.
- This means all memory locations assigned to a process are adjacent to one another.
- Main memory usually into two partitions:
 - ❑ Resident operating system, usually held in low memory with interrupt vector
 - ❑ User processes then held in high memory
 - ❑ Each process contained in single contiguous section of memory
- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - ❑ Base register contains value of smallest physical address
 - ❑ Limit register contains range of logical addresses – each logical address must be less than the limit register
 - ❑ MMU maps logical address *dynamically*
 - ❑ Can then allow actions such as kernel code being **transient** and kernel changing size

Hardware Support for Relocation and Limit Registers

- The **CPU** generates a **logical address**.
- The logical address is first compared with the **limit register**.
- The **limit register** stores the maximum valid logical address range for the process.
- So the hardware checks:

$$\text{Logical Address} < \text{Limit Register}$$

If the address is valid

The comparison gives **Yes**.

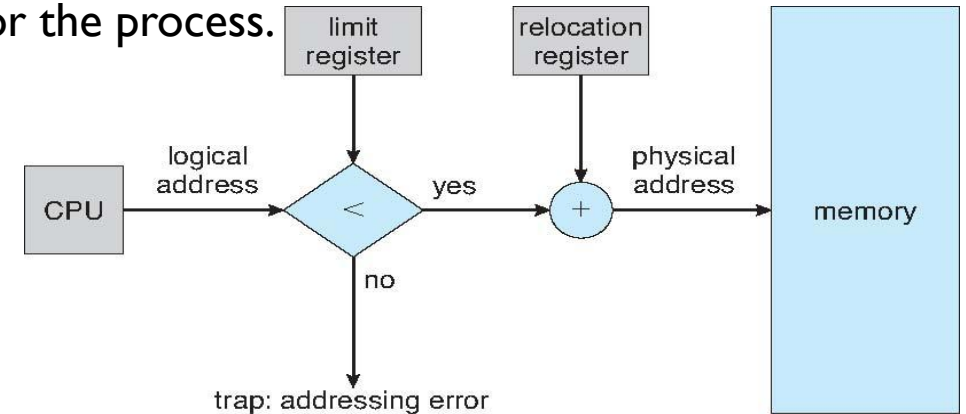
The logical address is added to the **relocation register**.

The relocation register stores the starting physical address of the process.

So:

$$\text{Physical Address} = \text{Relocation Register} + \text{Logical Address}$$

Then the generated physical address is sent to **main memory**.



If the address is invalid

If:

$$\text{Logical Address} \geq \text{Limit Register}$$

then the process is trying to access memory outside its permitted range.

The hardware generates a:

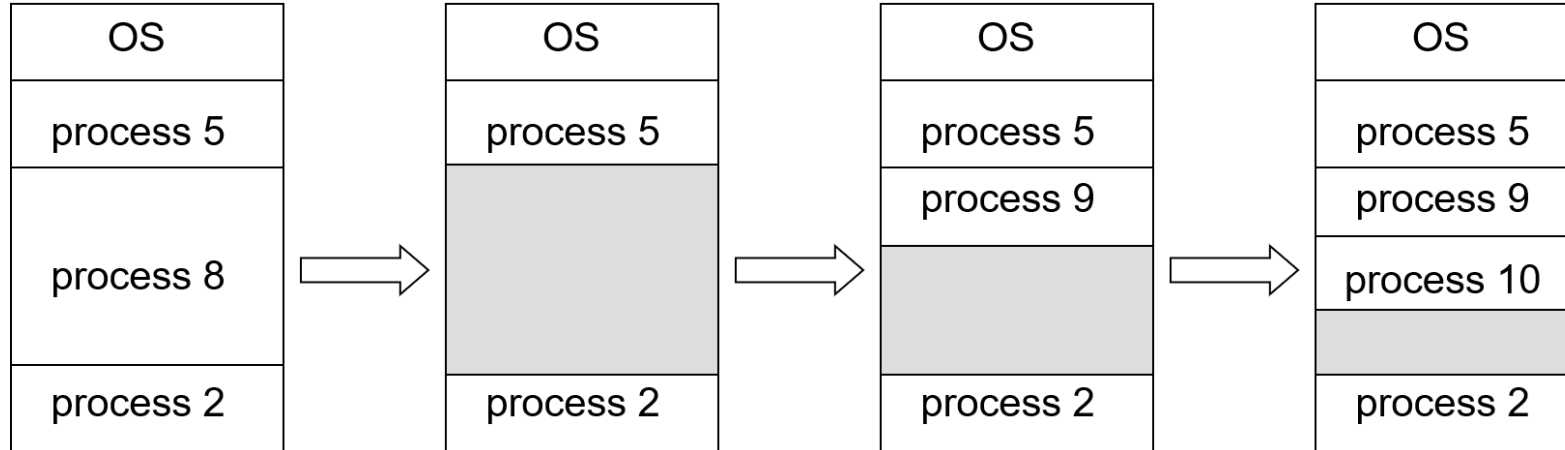
Trap → Addressing Error

and control is transferred to the operating system.

Hardware Support for Relocation and Limit Registers

Multiple-partition allocation

- Degree of multiprogramming limited by number of partitions
- Hole – block of available memory; holes of various size are scattered throughout memory
- When a process arrives, it is allocated memory from a hole large enough to accommodate it
- Process exiting frees its partition, adjacent free partitions combined
- Operating system maintains information about:
 - a) allocated partitions
 - b) free partitions (hole)



Memory Allocation Strategies

When a process needs memory, the operating system must choose a suitable free block, called a **hole**, from available main memory.

The three common allocation strategies are:

1. First Fit
2. Best Fit
3. Worst Fit

1. First Fit

The operating system selects the **first free block that is large enough** for the process.

Example: Free blocks: 100 KB, 500 KB, 200 KB, 300 KB

Process size: 180 KB

First Fit chooses: 500 KB

because it is the first block large enough.

Remaining space: $500 - 180 = 320$ KB

Advantages:

1. Simple, Fast
2. Requires less searching

Disadvantage:

1. May leave many small holes near the beginning of memory

Memory Allocation Strategies

2. Best Fit

The operating system selects the **smallest free block that is large enough** for the process.

Using the same free blocks: 100 KB, 500 KB, 200 KB, 300 KB

Process size: 180 KB

Best Fit chooses: 200 KB

Remaining space: $200 - 180 = 20$ KB

Advantages:

1. Tries to reduce wasted space in the selected block
2. Uses the smallest suitable hole

Disadvantages:

1. Must often search the entire list
2. Can create many very small unusable holes

Memory Allocation Strategies

3. Worst Fit

The operating system selects the **largest available free block**.

Example:

Free blocks: 100 KB, 500 KB, 200 KB, 300 KB

Process size: 180 KB

Worst Fit chooses: 500 KB

Remaining space: $500 - 180 = 320$ KB

Advantages:

1. Leaves a relatively large free block
2. Avoids creating very tiny holes in some cases

Disadvantages:

1. May waste large blocks
2. Usually less efficient than First Fit and Best Fit

Strategy	Selection Rule	Search Speed	Main Issue
First Fit	First suitable hole	Fast	Small holes may accumulate
Best Fit	Smallest suitable hole	Slower	Creates tiny unusable holes
Worst Fit	Largest available hole	Slower	Large blocks may be wasted

Illustration

Free memory blocks: 120 KB, 400 KB, 250 KB, 600 KB

Process requires: 230 KB

First Fit → 400 KB

Best Fit → 250 KB

Worst Fit → 600 KB

Dynamic Storage-Allocation Problem

Advantages of Contiguous Allocation

- Simple to understand and implement.
- Address translation is straightforward.
- Fast memory access.
- Requires relatively little memory-management overhead.

Disadvantages

- Suffers from fragmentation.
- Fixed partitions cause **internal fragmentation**.
- Variable partitions cause **external fragmentation**.
- Process size may be limited by the largest available contiguous block.
- Compaction may be required.

Fragmentation

Fragmentation is the wastage of memory that occurs when available memory cannot be used efficiently. It mainly occurs during memory allocation and deallocation.

First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation

$1/3$ may be unusable -> **50-percent rule**

There are two major types:

- 1. Internal Fragmentation**
- 2. External Fragmentation**

Fragmentation

I. Internal Fragmentation

It occurs when a process is allocated a memory block that is larger than what it actually needs.

The unused memory remains **inside the allocated block**

Example : Suppose,

Partition size = **100 KB**

Process size = **70 KB**

Then: $100 - 70 = 30 \text{ KB}$

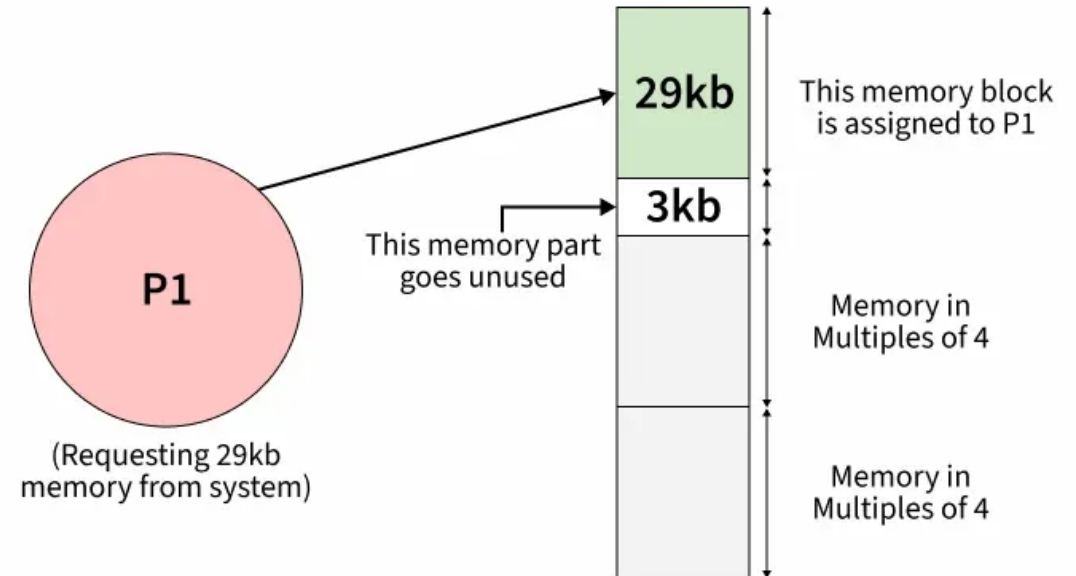
The remaining **30 KB** is wasted.

Causes of Internal Fragmentation

- Fixed-size memory partitions
- Fixed-size pages or blocks
- Allocating a block larger than the actual process requ
- Memory allocation granularity

Solutions

- Use smaller allocation units
- Use variable-size partitions
- Use better memory-allocation techniques
- In paging, use a suitable page size
- However, internal fragmentation cannot always be eliminated completely.



Fragmentation

2. External Fragmentation

It occurs when free memory is available, but it is divided into many small non-contiguous holes.

The total free memory may be enough for a process, but there may not be one sufficiently large continuous block.

Example

Suppose free memory blocks are:

20 KB

30 KB

40 KB

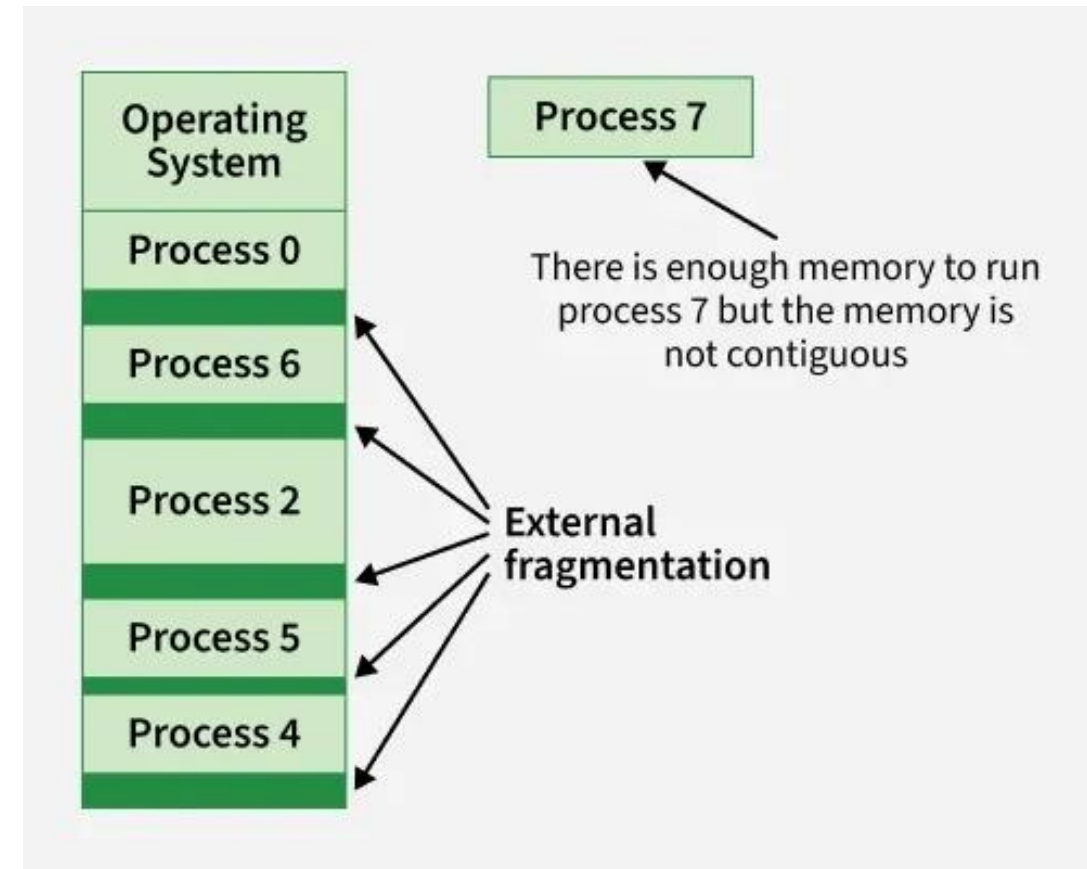
Total free memory: $20 + 30 + 40 = 90 \text{ KB}$

A new process requires: **70 KB**

Although **90 KB is free**, the process cannot be allocated because there is **no single contiguous block of 70 KB**.

Causes of External Fragmentation

1. Variable-size contiguous memory allocation
2. Processes entering and leaving memory repeatedly
3. Different process sizes
4. Repeated splitting of free memory blocks
5. First-fit, best-fit, or worst-fit allocation may leave scattered holes



Solutions for External Fragmentation

1. Compaction

Move processes so that small free holes are combined into one large free block.

Before compaction:

P1 | Free | P2 | Free | P3 | Free

After compaction:

P1 | P2 | P3 | Large Free Block

Problem: Compaction is costly because processes must be relocated.

2. Paging

- Divide logical memory into fixed-size **pages**
- Divide physical memory into fixed-size **frames**
- Pages can be placed in non-contiguous frames
- Therefore, a process does not require one continuous block of memory.
- Paging largely **removes external fragmentation**.

3. Segmentation with Paging

- Segments are divided into pages.
- Combines the benefits of segmentation and paging.
- Reduces external fragmentation.

Paging

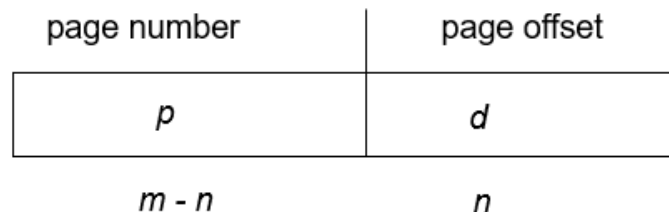
- Paging solves this problem by dividing memory into fixed-size blocks.
- **Logical memory** of a process is divided into fixed-size blocks called **pages**.
- **Physical memory (RAM)** is divided into fixed-size blocks called **frames**.
- The size of a page and frame is always the same.

Page Size = Frame Size

- Backing store likewise split into pages
- Still have Internal fragmentation

Address Translation Scheme:

- Address generated by CPU is divided into:
 - **Page number (p)** – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit



- For given logical address space 2^m and page size 2^n

Paging Model of Logical and Physical Memory

Logical Memory (Virtual Memory)

- The memory view seen by a process is called **logical memory**.
- It is divided into pages.

Example:

- Logical Address Space of Process P1
- The process believes its pages are arranged continuously.

Physical Memory

- The actual RAM is divided into frames.

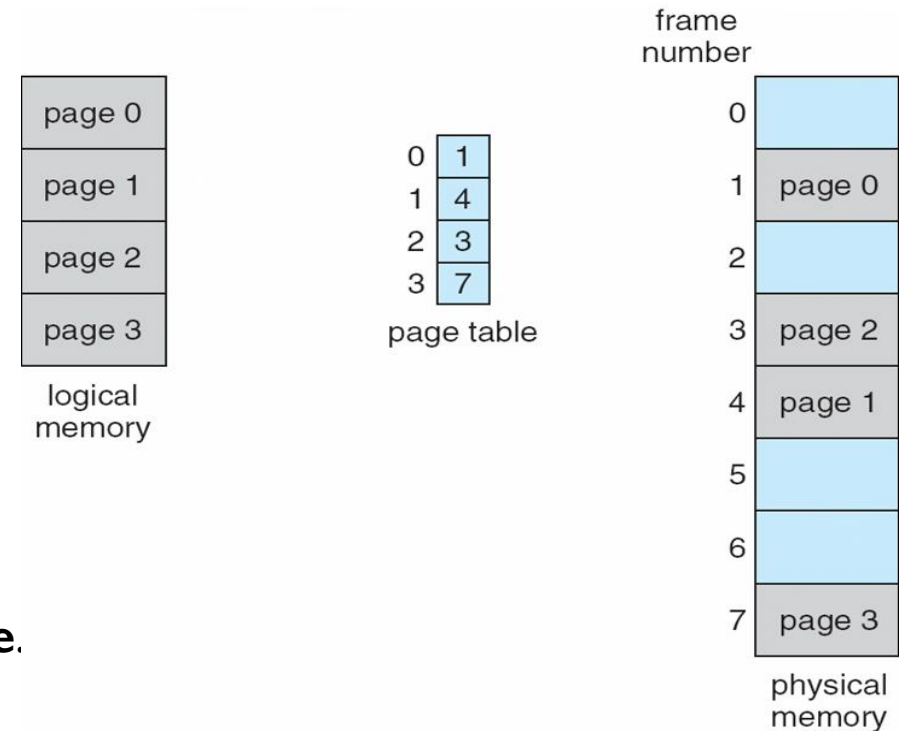
Page to Frame Mapping

- Pages can be stored in any available frames.
- The process does not know where its pages are physically stored.

Page Table

- The operating system maintains a data structure called a **page table**.
- The page table stores the mapping:
- Page Number → Frame Number

- In the above example, page 0 is loaded to frame 1, page 1 to frame 4, page 2 to frame 3 and page 3 to frame 7



Address Translation in Paging

- The CPU generates a **logical address**. page number + offset
 - Page Number** : Identifies the page containing the required data.
 - Offset** : Identifies the exact location within that page.

Example

Assume: Page Size = 1024 bytes

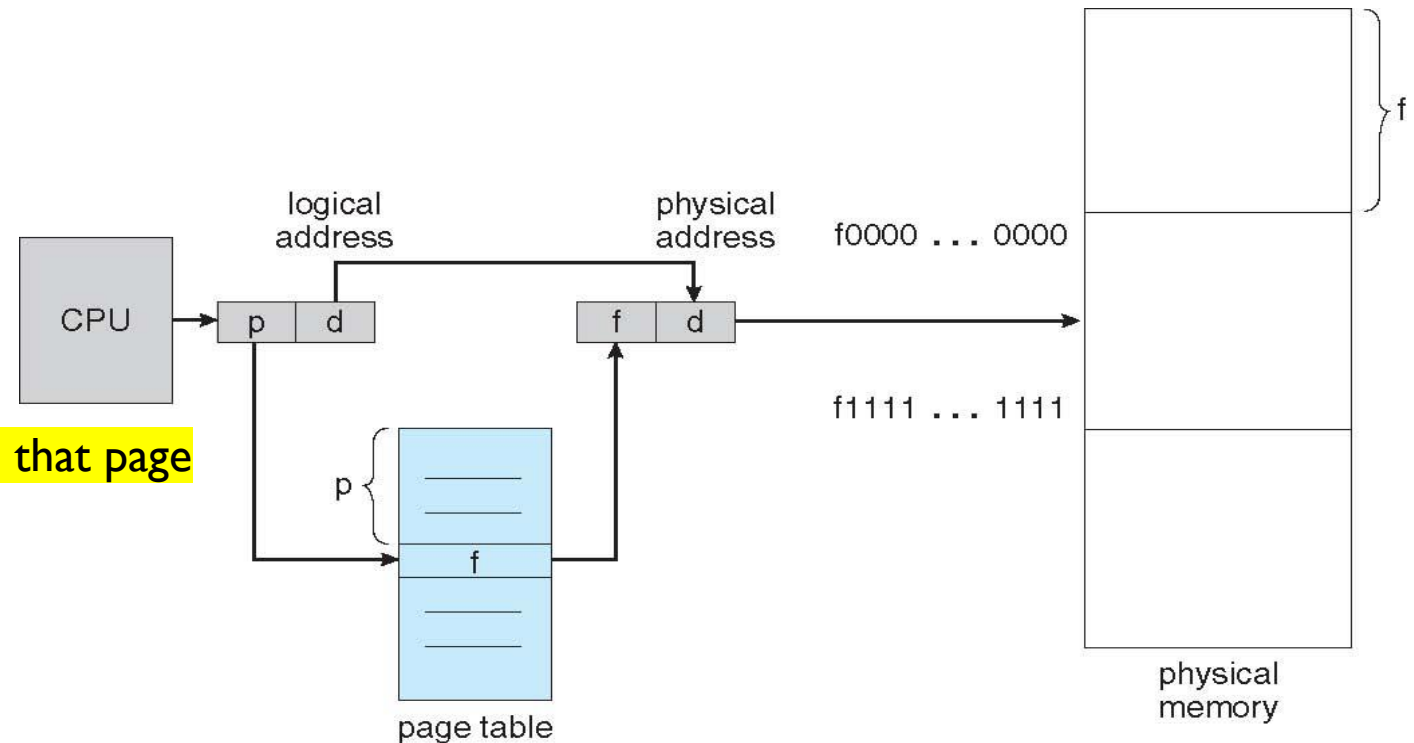
Logical Address = 2500

The address is divided into:

- Page Number** $2500/1024 = 2$
- Offset** $2500 - (2 \times 1024) = 452$

Therefore: Page Number = 2. Offset = 452

Meaning: Access Page 2 and Go 452 bytes inside that page



Address Translation in Paging

Example :

1. CPU generates:

- Page Number = 2, Offset = 452
- Search page table: Page 2 → Frame 5

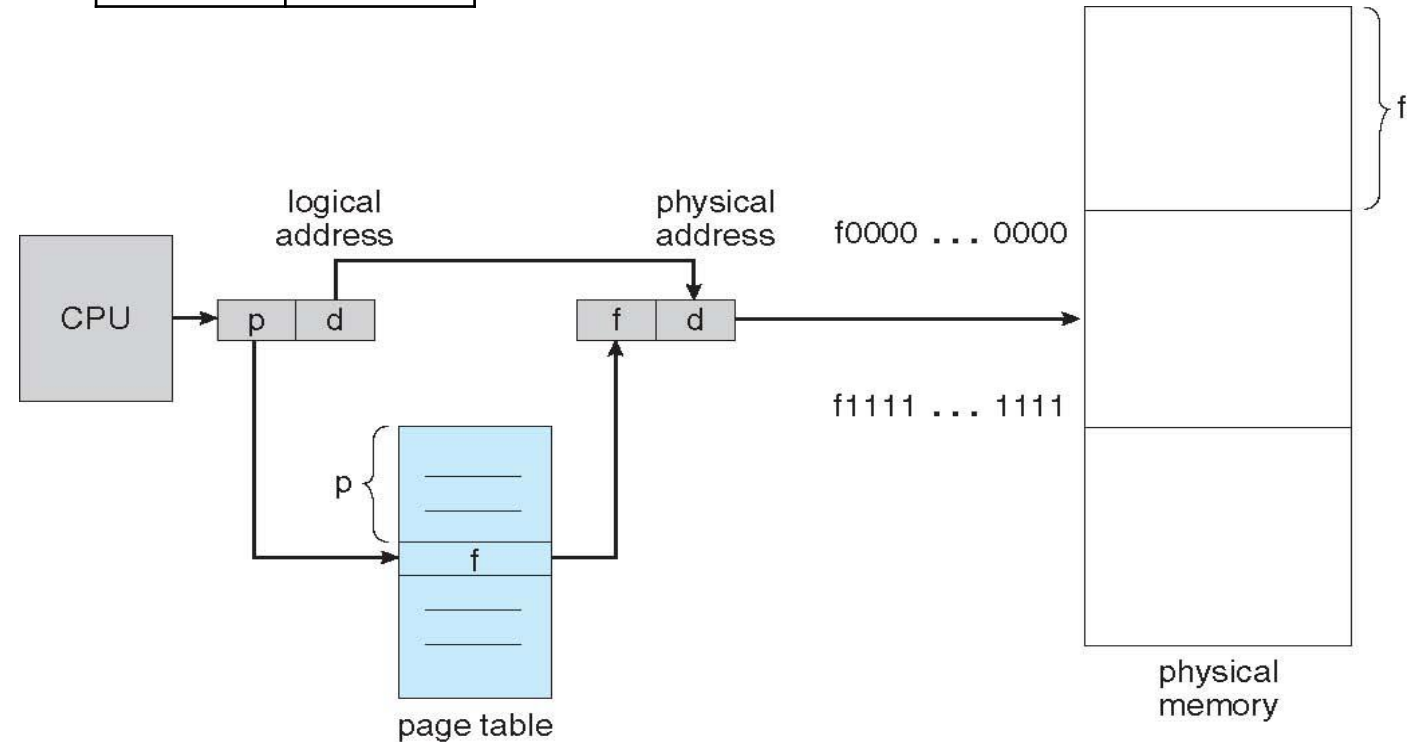
2. Generate physical address:

- Frame Number = 5
- Offset = 452
- Physical Address = Frame 5 + Offset 452

3. The CPU accesses the data in:

- **Frame 5, location 452**

Page	Frame
0	3
1	7
2	5
3	1



Paging Model of Logical and Physical Memory

Locate where the data with logical address 9 is loaded to the memory?

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Solution :

Page no = $9/4 = 2$ page

Offset = $9\%4 = 1$

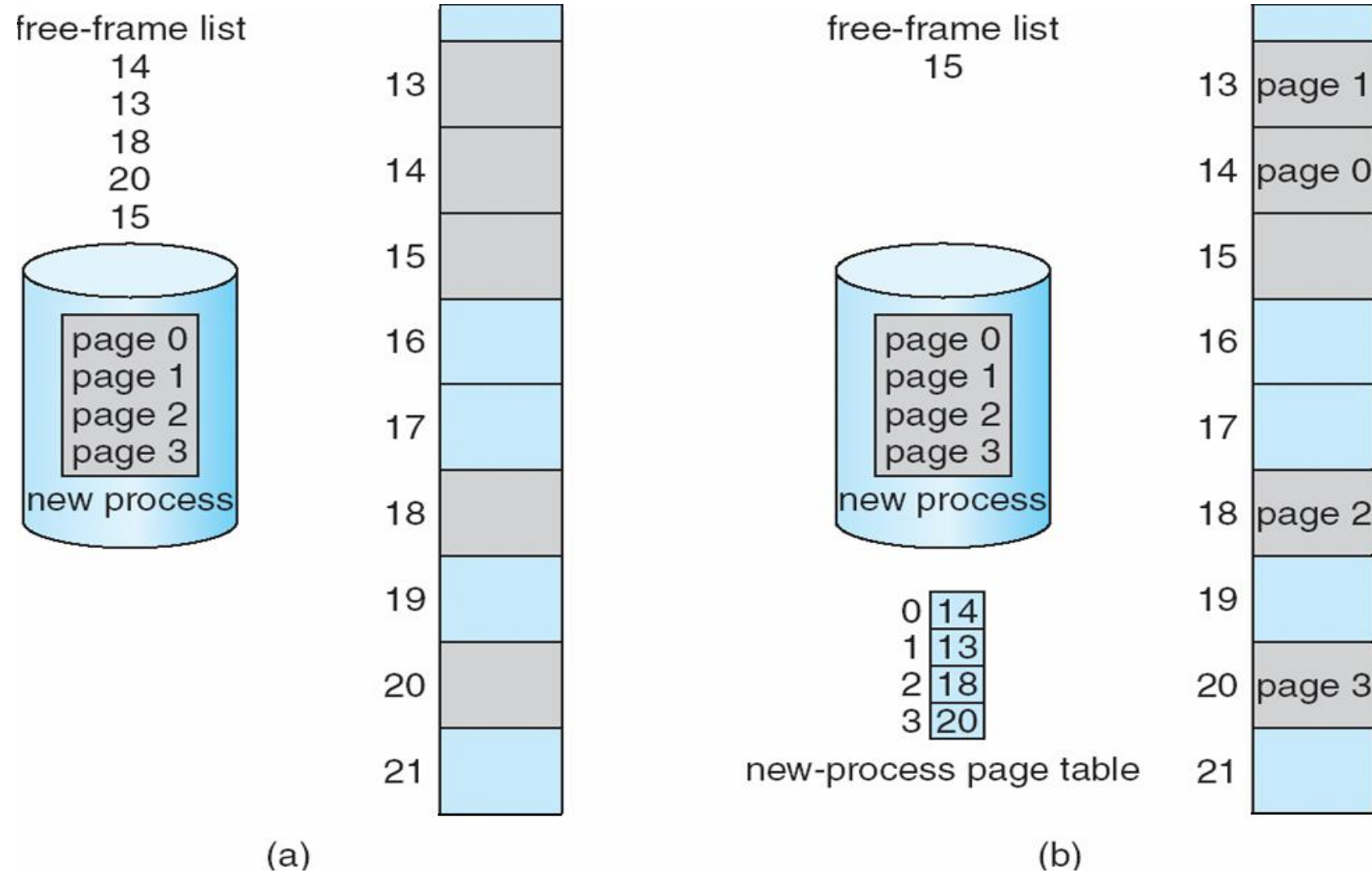
Check table page 2 is indexed to frame 1

Physical address = frame no+ offset

$9 \rightarrow (2,1) \rightarrow (1,1) \rightarrow 5$

The page number changes to a frame number; the offset remains unchanged.

Paging Model of Logical and Physical Memory



Structure of Page table

- A page table is made up of **Page Table Entries (PTEs)**.

- A simplified PTE looks like: **Frame no | control/status info** eg :

Frame no	Valid	R/W/X	Ref.	Dirty
----------	-------	-------	------	-------

- The control and status information can contain several bits.

2. Valid/Invalid Bit - Indicates whether the page is a valid part of the process's address space. 1/0 –valid/invalid

3. Present/Absent Bit - Indicates whether the page is currently in physical memory.

Present = 1 → Page is in RAM

Present = 0 → Page is not currently in RAM, If the page is not present and is required, a **page fault** occurs.

4. Protection Bits - Specify what operations are allowed.

- R → Read
- W → Write
- X → Execute

Example:

- Code page → R-X
- Data page → RW-

Structure of Page table

- A page table is made up of **Page Table Entries (PTEs)**.

- A simplified PTE looks like: **Frame no | control/status info** eg :

Frame no	Valid	R/W/X	Ref.	Dirty
----------	-------	-------	------	-------

- The control and status information can contain several bits.

2. Valid/Invalid Bit - Indicates whether the page is a valid part of the process's address space. 1/0 –valid/invalid

3. Present/Absent Bit - Indicates whether the page is currently in physical memory.

Present = 1 → Page is in RAM

Present = 0 → Page is not currently in RAM, If the page is not present and is required, a **page fault** occurs.

4. Protection Bits - Specify what operations are allowed.

- R → Read
- W → Write
- X → Execute

Example:

- Code page → R-X
- Data page → RW-

Structure of Page table

5. Reference Bit - Indicates whether the page has been accessed.

- It is useful for page replacement algorithms.
- Reference = 1 $\rightarrow$ Page has been recently accessed

6. Dirty/Modified Bit - Indicates whether the contents of the page have been modified after being loaded into memory.

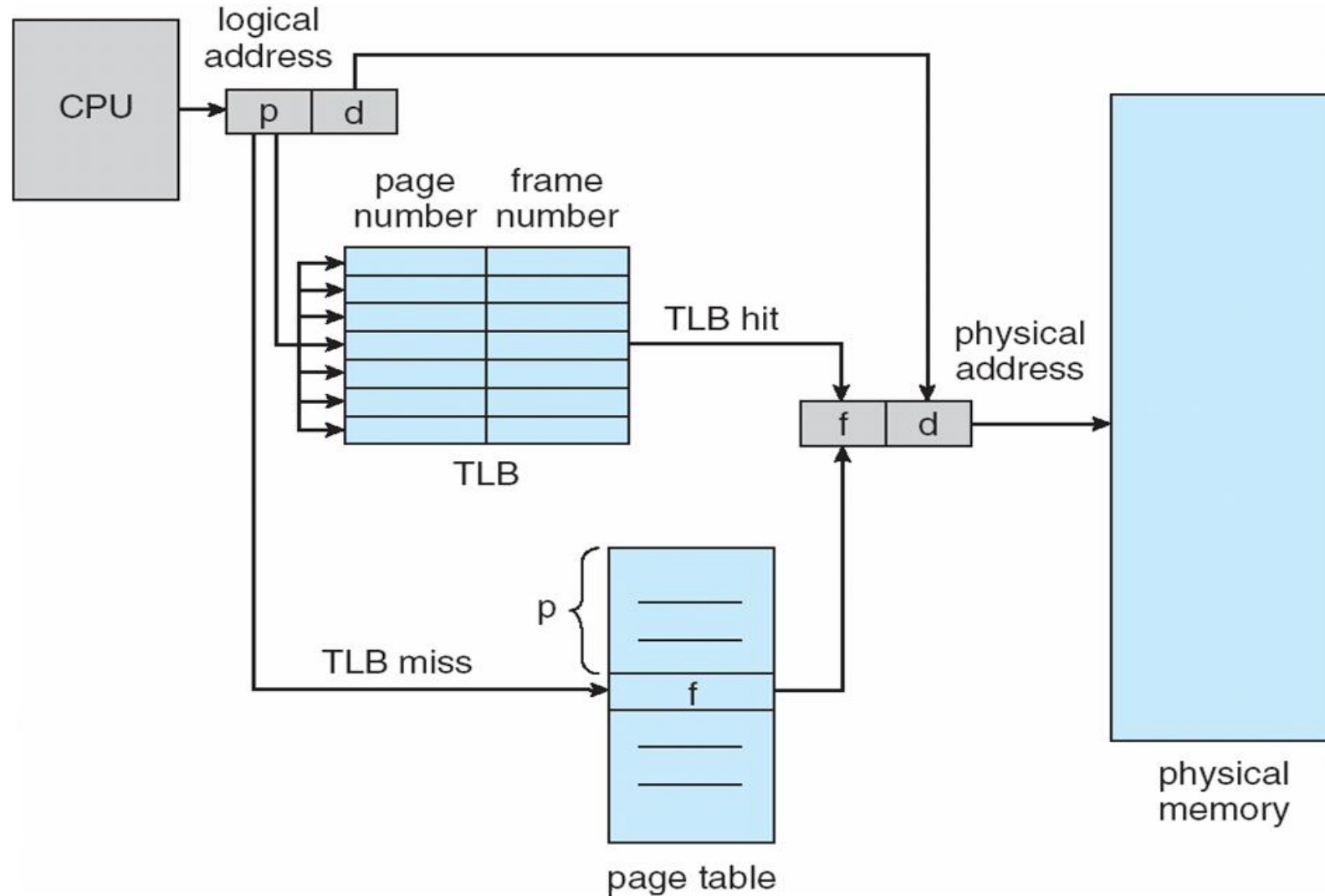
- Dirty = 0 $\rightarrow$ Not modified
- Dirty = 1 $\rightarrow$ Modified

A modified page may need to be written back to disk before it is replaced.

Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PRLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**
- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
 - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
 - Some entries can be **wired down** for permanent fast access

Paging Hardware With TLB

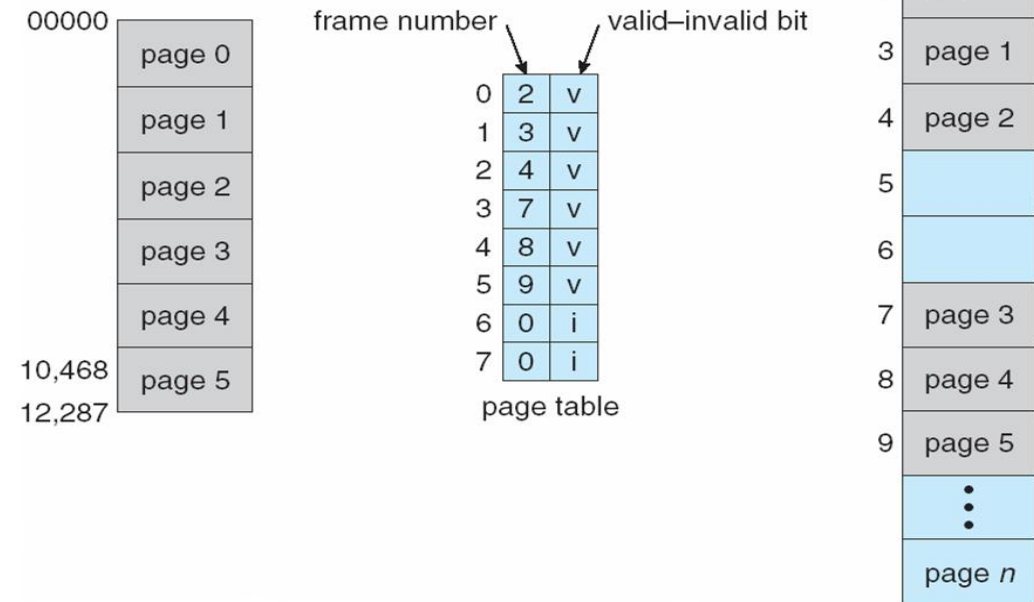


Effective Access Time

- Associative Lookup = ε time unit
 - Can be $< 10\%$ of memory access time
- Hit ratio = α
 - Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers
- Consider $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ for TLB search, 100ns for memory access
- Effective Access Time (EAT)
$$\begin{aligned}\text{EAT} &= (1 + \varepsilon) \alpha + (2 + \varepsilon)(1 - \alpha) \\ &= 2 + \varepsilon - \alpha\end{aligned}$$
- Consider $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ for TLB search, 100ns for memory access
 - $\text{EAT} = 0.80 \times 120 + 0.20 \times 220 = 140\text{ns}$
- Consider slower memory but better hit ratio $\rightarrow \alpha = 98\%$, $\varepsilon = 20\text{ns}$ for TLB search, 140ns for memory access
 - $\text{EAT} = 0.98 \times 160 + 0.02 \times 300 = 162.8\text{ns}$

Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
 - Can also add more bits to indicate page execute-only, and so on
- Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use PTLR
- Any violations result in a trap to the kernel



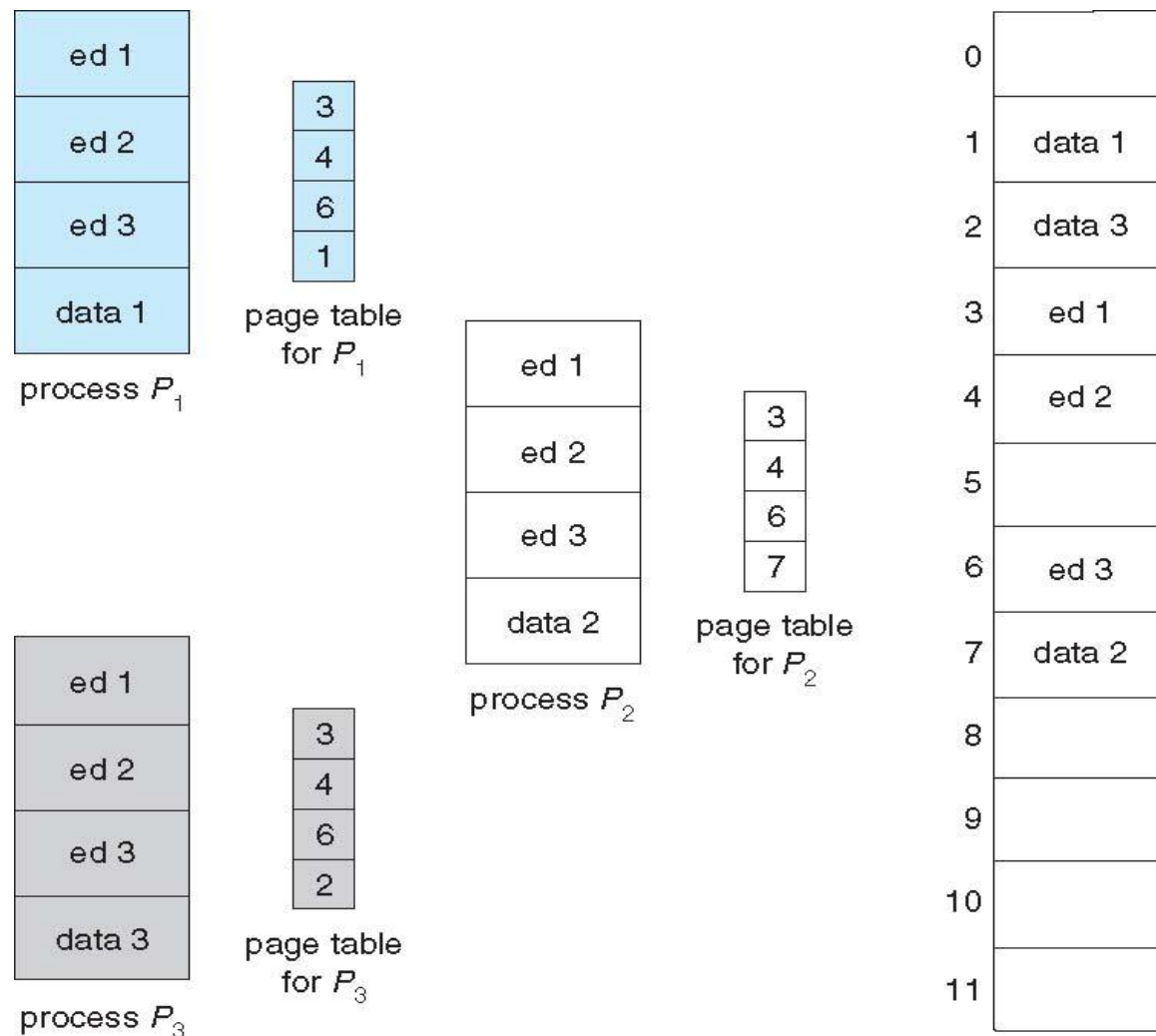
Shared Pages

- **Shared code**

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

- **Private code and data**

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space



Types of Page Tables

- Memory structures for paging can get huge using straight-forward methods
 - Consider a 32-bit logical address space as on modern computers
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone
 - That amount of memory used to cost a lot
 - Don't want to allocate that contiguously in main memory

Types of Page Table

1. Hierarchical Paging
2. Hashed Page Tables
3. Inverted Page Tables

Types of Page Tables

1. Single-Level Page Table

- This is the simplest form.
- There is **one page table for each process**, with one entry for each virtual page.

Page no	Frame no
0	3
1	8
2	0

Advantage

- Simple
- Direct lookup

Disadvantage

- For a large virtual address space, the page table can become **very large**.
- This is the main reason multilevel, hashed, and inverted page tables are used.

2. Multilevel / Hierarchical Page Table:

- Instead of maintaining one very large page table, the page table is divided into multiple levels.
- For a two-level page table, the virtual address is divided into:

Outer Page no	Inner Frame no	offset
0	3	
1	8	
2	0	

Types of Page Tables

Why use it multilevel paging?

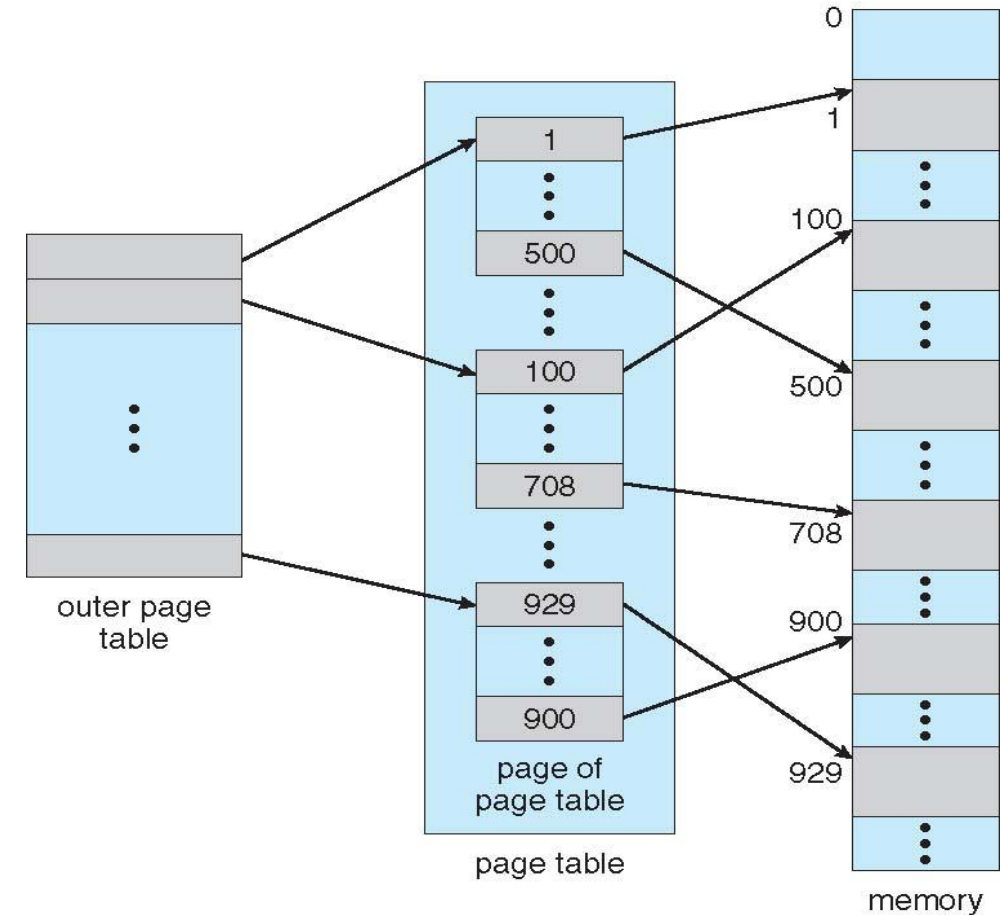
- A large address space may contain huge unused regions.
- Instead of creating entries for all possible pages, the OS creates lower-level page tables only for portions that are actually needed.

Advantage

- Reduces page-table memory usage.

Disadvantage

- More levels mean more memory accesses for translation if the required entries are not cached in the TLB.



Outer Page no	Inner Frame no	offset
0	3	
1	8	
2	0	

Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
 - a page number consisting of 22 bits
 - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
 - a 12-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:

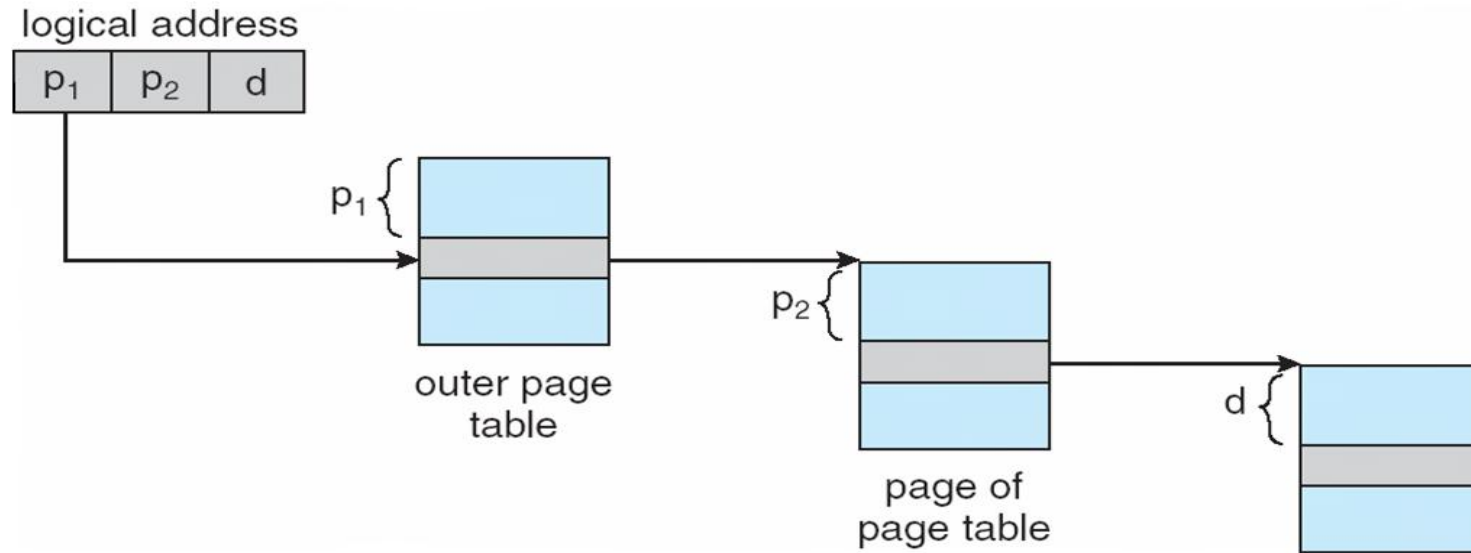
page number		page offset
p_1	p_2	d
12	10	10

- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the outer page table

Known as **forward-mapped page table**

Two-Level Paging Example

Address-Translation Scheme



Types of Page Tables

3.. Hashed Page Table

- Hashed page tables are particularly useful for **large address spaces**, such as 64-bit systems.
- Instead of directly indexing a huge page table, the virtual page number is passed through a **hash function**.
- The virtual page number is hashed into a page table
 - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
 - If a match is found, the corresponding physical frame is extracted

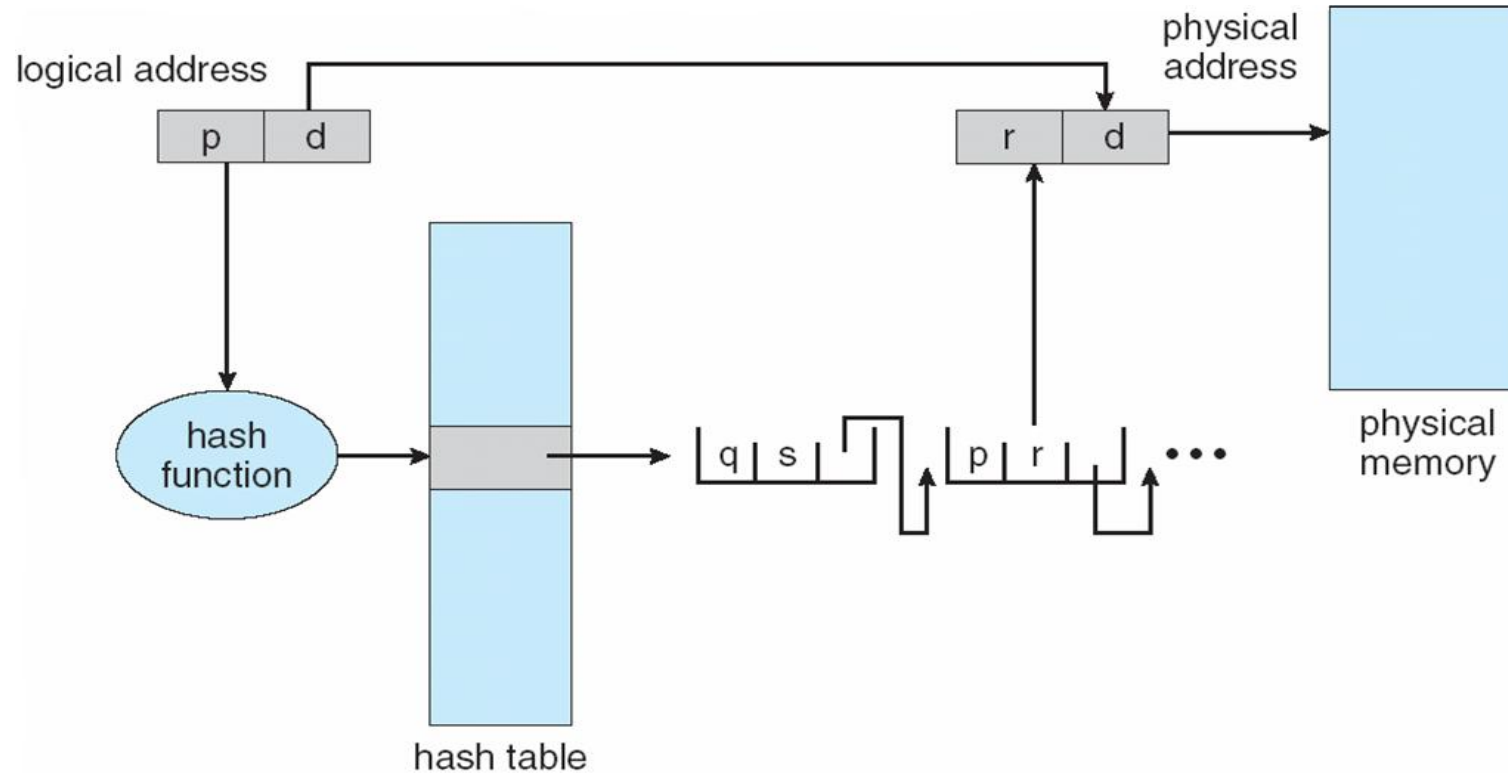
Advantage

- Avoids maintaining an enormous entry for every possible virtual page.
- Suitable for very large address spaces.

Disadvantage

- Hash collisions require additional searching.

Hashed Page Table



Inverted Page Table

4. Inverted Page Table

A conventional page table has an entry for **each virtual page**.

- An inverted page table has an entry for **each physical frame**.
- Instead of: **Virtual Page → Frame**
- it maintains information such as: **Physical Frame → Process ID + Virtual Page**
- There is only **one entry per physical frame**.

Address translation

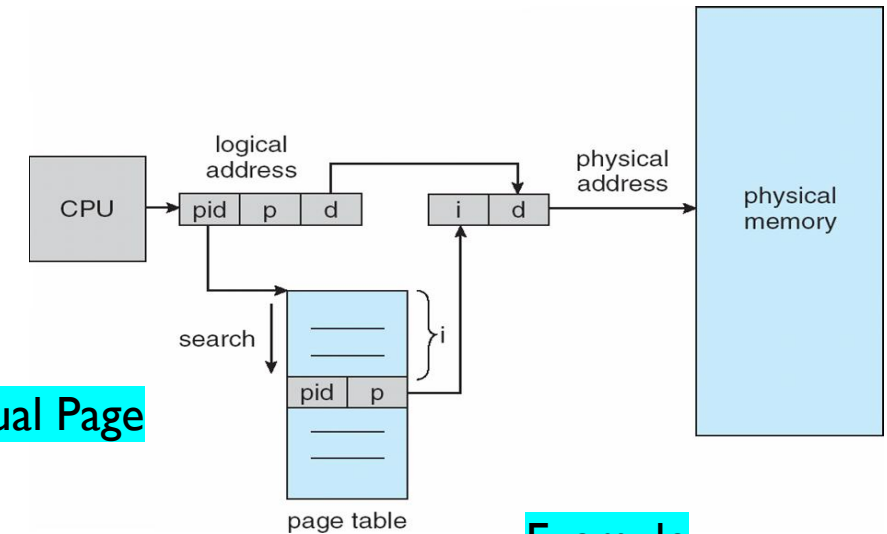
- The system searches for the combination:
- Process ID + Page Number
- and finds the corresponding frame, A hash table is often used to make this lookup

Advantage

- Considerably reduces page-table memory overhead for very large virtual address spaces.

Disadvantage:

- Lookup is more complicated than direct indexing.
- Usually requires hashing or other hardware support for efficient searching.



Example

Frame	Process id	Page no
0	p2	3
1	p1	2
2	p3	1
3	p1	5

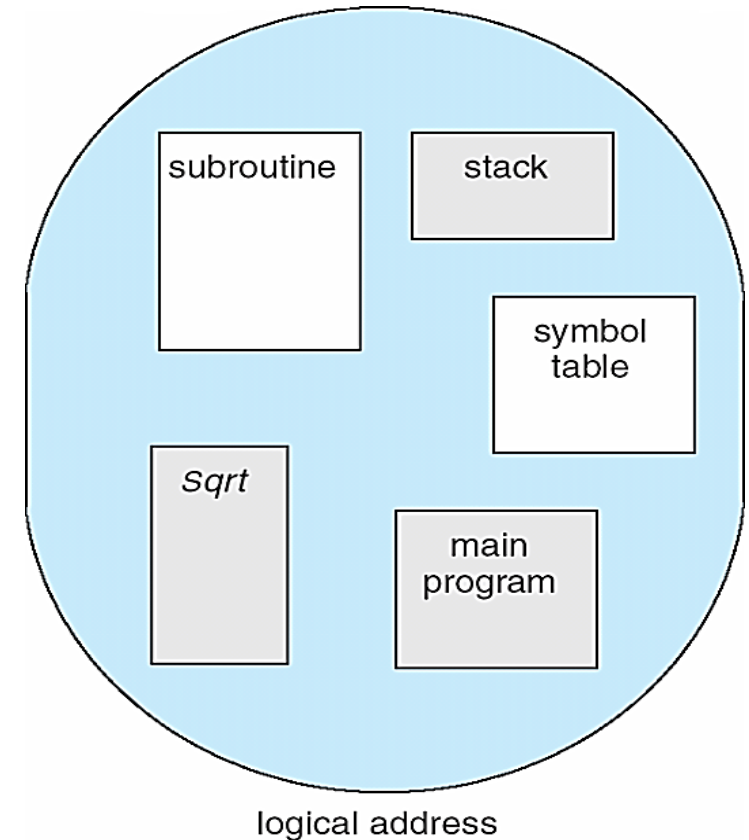
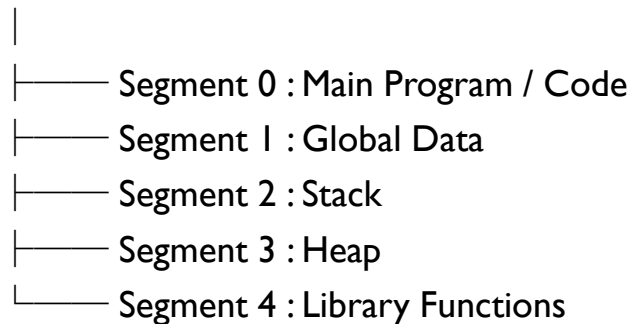
Segmentation

Segmentation is a memory-management technique in which a program is divided into **logical, variable-sized units called segments**.

- Typical segments include:
 - Code segment
 - Data segment
 - Stack segment
 - Heap segment
 - Functions / procedures
 - Arrays or tables
- Unlike paging, segmentation follows the **logical structure of a program**.

Basic idea : Consider a program consisting of:

Program



- **These segments need not be stored next to each other in physical memory.**

Segmentation

Logical address in segmentation

- A logical address contains two parts:
- Logical Address= $\langle$ Segment Number, Offset $\rangle$
- For example: $\langle 2, 450 \rangle$, means: Segment number = **2** , Offset = **450**

Therefore, the CPU is requesting the memory location located **450 bytes from the beginning of Segment 2.**

Segment Table

- The operating system maintains a **segment table** for every process.
- Each entry normally contains:

Base → starting physical address of the segment

Limit → size of the segment

Segment	Purpose	Base	Limit
0	Code	1000	600
1	Data	4000	800
2	Stack	7000	500
3	Heap	9000	1000

Segment-table base register (STBR) points to the segment table's location in memory

Segment-table length register (STLR) indicates number of segments used by a program;

segment number **s** is legal if **s** < **STLR**

Segmentation

Address Translation

Suppose the CPU generates:

$\langle 1, 300 \rangle$

From the segment table:

Segment 1

Base = 4000

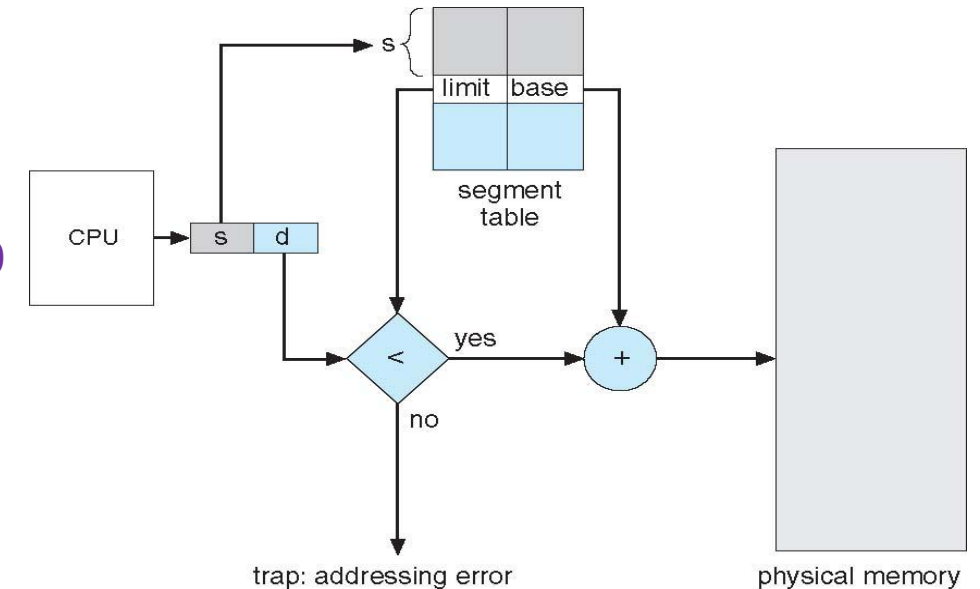
Limit = 800

First, the MMU checks: $Offset < Limit$ *i.e* $300 < 800$

Valid address.

The physical address is: $Physical\ Address = Base + Offset$
 $= 4000 + 300$
 $= 4300$

Segment	Purpose	Base	Limit
0	Code	1000	600
1	Data	4000	800
2	Stack	7000	500
3	Heap	9000	1000



Segmentation

Example Problem

Given the segment table:

Find the physical addresses of:

- a. <0, 430>
- b. <1, 10>
- c. <2, 500>
- d. <3, 400>

Segment	Base	Limit
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

Protection

With each entry in segment table associate:

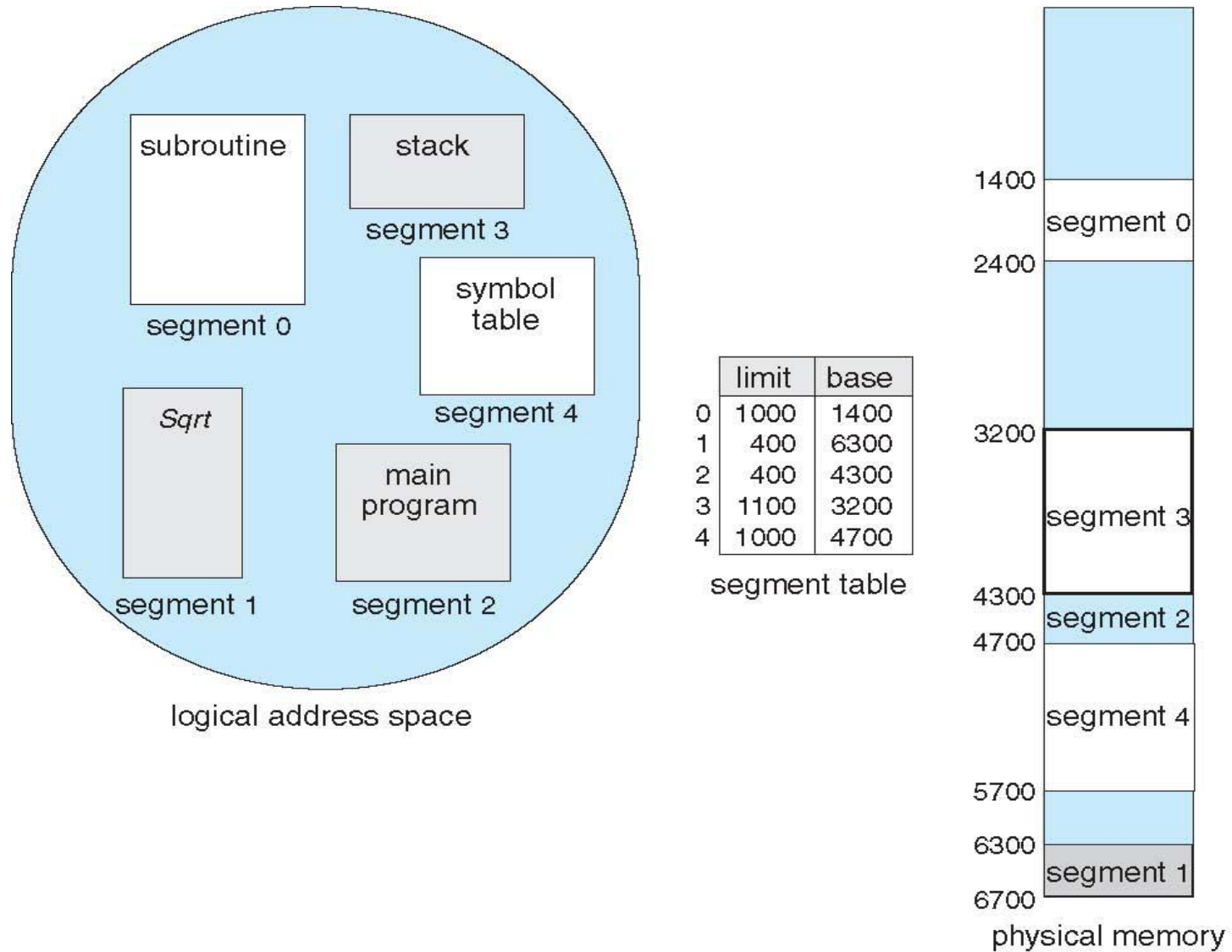
validation bit = 0 $\Rightarrow$ illegal segment

read/write/execute privileges

Protection bits associated with segments; code sharing occurs at segment level

Since segments vary in length, memory allocation is a dynamic storage-allocation problem

Segmentation Example



Summary - Segmentation

Since segments have **variable sizes**, segmentation can cause **external fragmentation**.

Advantages of Segmentation

- Represents the **logical structure** of a program.
- Different segments can have different sizes.
- Supports **protection** at the segment level.
- Supports **sharing** of code and libraries.
- Makes modular programming easier.
- Code, stack, data and heap can be managed separately.

Disadvantages

- Causes **external fragmentation**.
- Allocation is more complicated because segment sizes vary.
- Compaction may be required.
- Segment-table management introduces additional overhead.

Feature	Segmentation	Paging
Division	Logical units	Fixed-size blocks
Size	Variable	Fixed
Address	Segment + Offset	Page + Offset
Programmer's view	Visible/logical	Usually invisible
Fragmentation	External	Internal
Protection/sharing	Natural at segment level	Page-level
Example units	Code, stack, data	Pages of 4 KB etc.

Demand Paging

- **Demand paging** is a virtual-memory technique in which a page is loaded into main memory **only when the process actually needs that page**.
- Instead of loading the entire process into RAM before execution, the OS initially loads only the required pages. Other pages remain on disk until referenced.
- Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory

- **Lazy swapper** – never swaps a page into memory

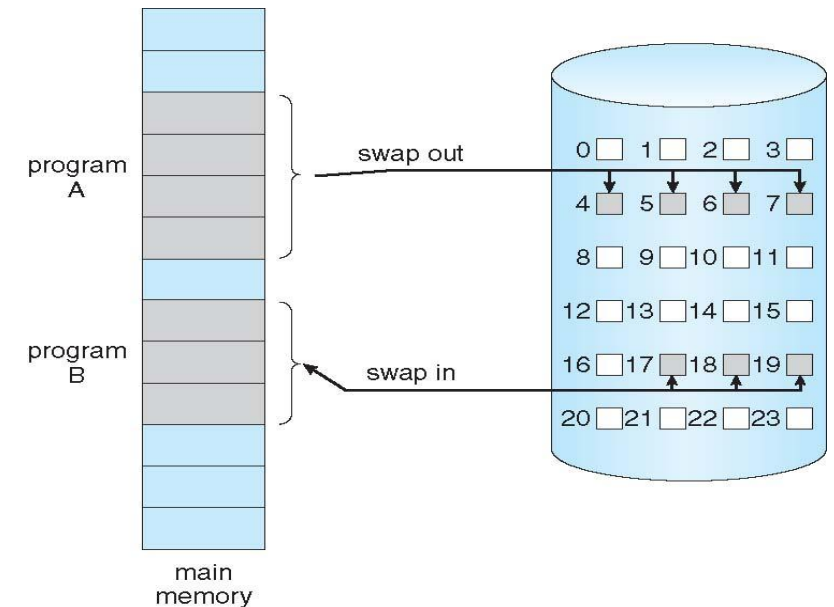
unless page will be needed

- Swapper that deals with pages is a **pager**

Suppose a process contains 8 pages:

- With normal loading, all pages may be brought into memory.
- With **demand paging**, perhaps only Pages 0, 2, and 5 are currently required:

The remaining pages are loaded **when demanded**.



Demand Paging

How does the OS know whether a page is in memory?

- Each page-table entry contains a **valid/invalid bit**, sometimes called the **present bit**.
- **Valid** → page is currently present in physical memory.
- **Invalid** → page is not currently present in memory.
- Suppose the CPU tries to access **Page 3**.
- The page table shows:
- Page 3 → Invalid
- This causes a: Page Fault

Page	Frame	Valid/Invalid
0	4	Valid
1	—	Invalid
2	7	Valid
3	—	Invalid
4	—	Invalid
5	2	Valid

A **page fault** is an exception generated when a process accesses a page that is not currently in main memory.

Demand Paging

Example:

- Assume the CPU generates logical address:
- Page number = 4 ; Offset = 200
- For Page 4: Valid bit = 1

Page	Frame	Valid/Invalid
0	3	V
1	6	V
2	--	i
3	1	v
4	—	i

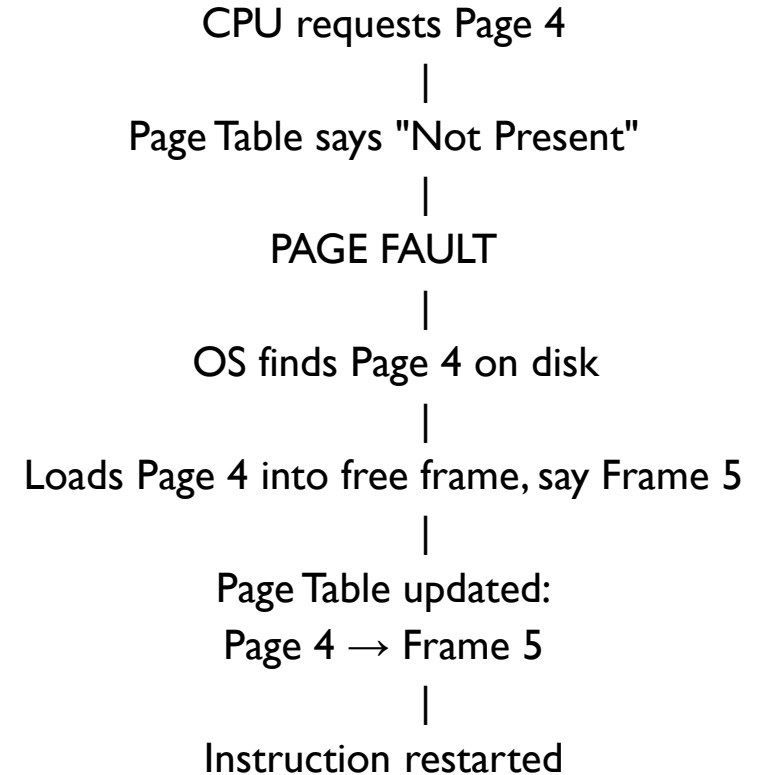
Because pages are brought into RAM **only on demand**.

For example, a large program may contain 100 pages, but during a particular execution it may use only 30 pages.

Without demand paging: 100 pages → loaded into RAM

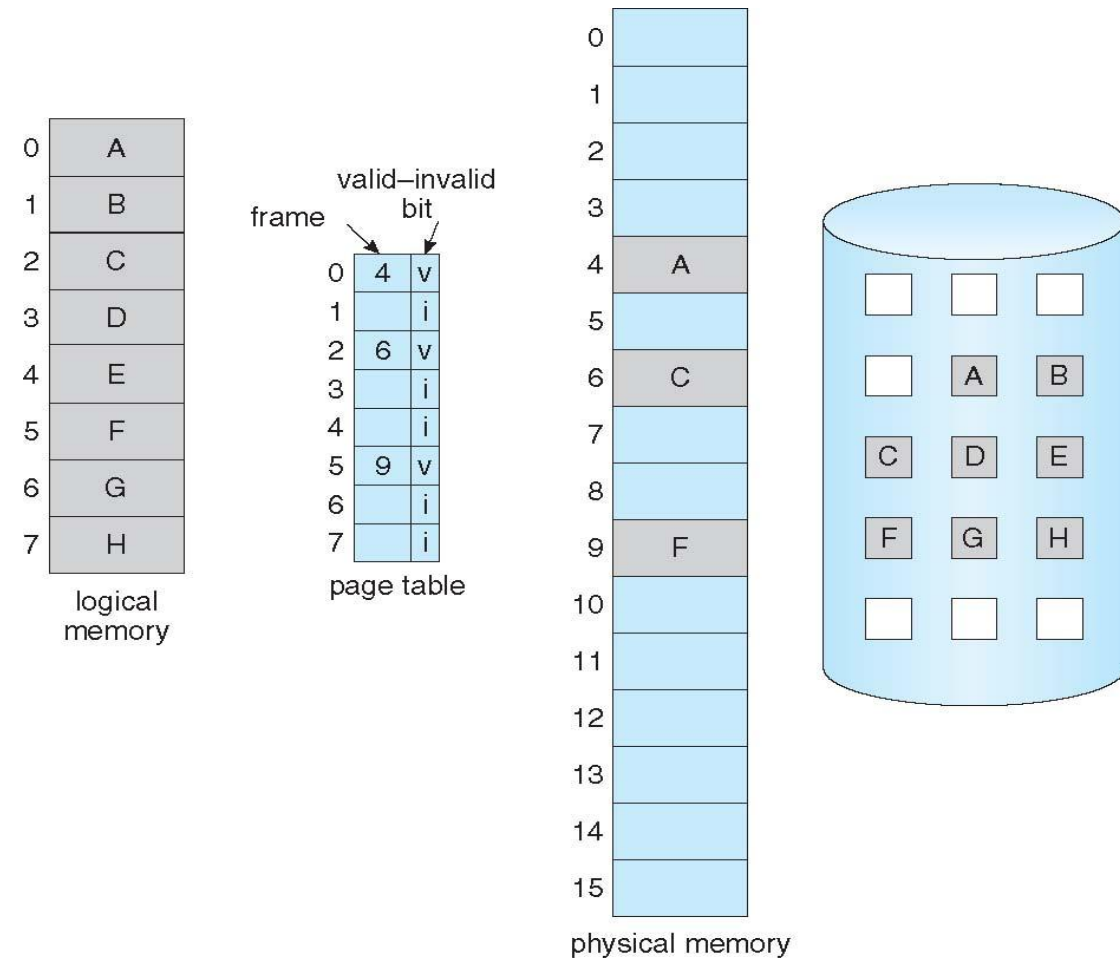
With demand paging: Only required 30 pages → loaded into RAM

Therefore memory is used more efficiently.

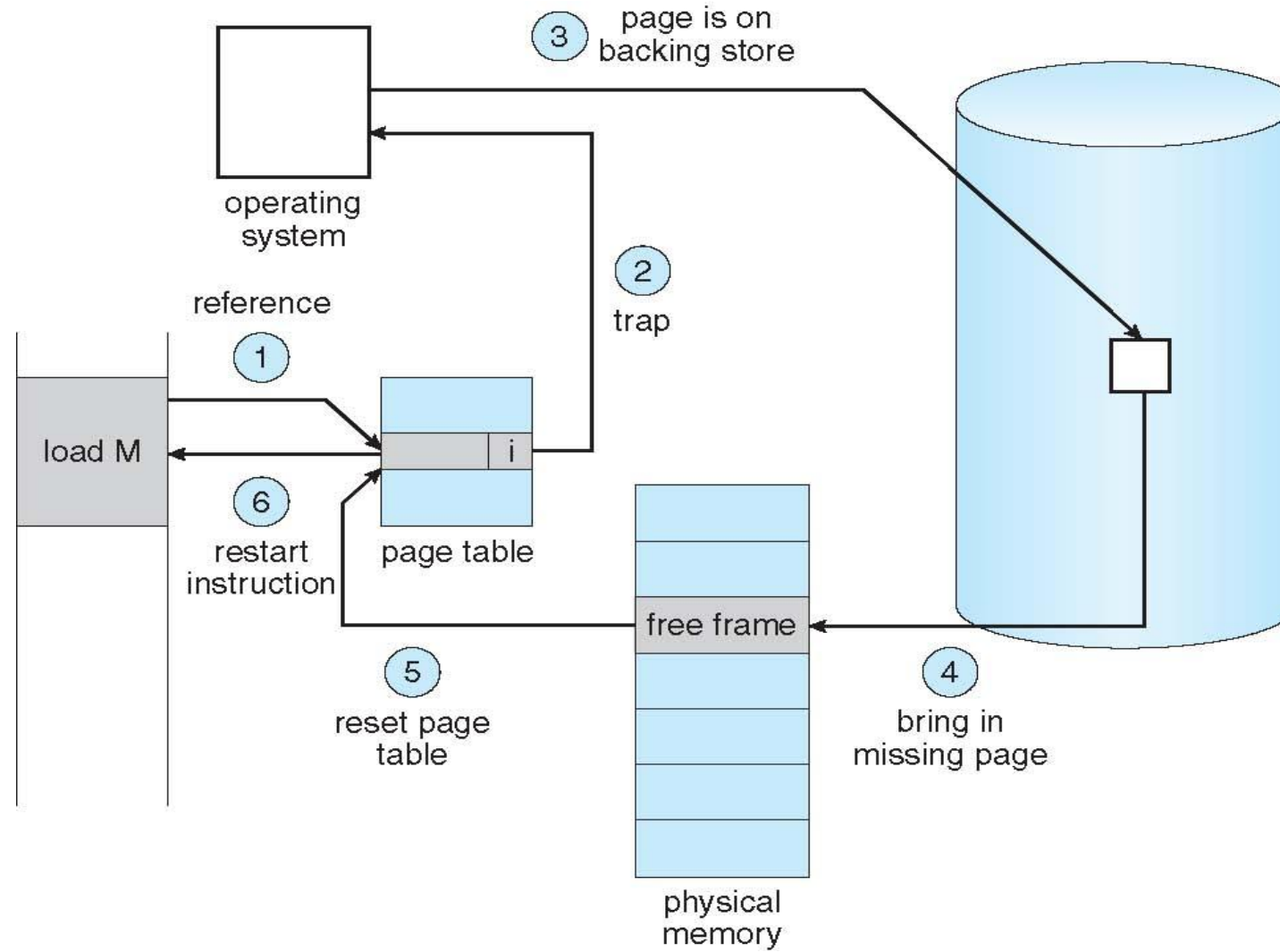


Demand Paging

- Extreme case – start process with *no* pages in memory
 - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
 - And for every other process pages on first access
 - **Pure demand paging**
- Actually, a given instruction could access multiple pages -> multiple page faults
 - Pain decreased because of **locality of reference**
- Hardware support needed for demand paging
 - Page table with valid / invalid bit
 - Secondary memory (swap device with **swap space**)
 - Instruction restart



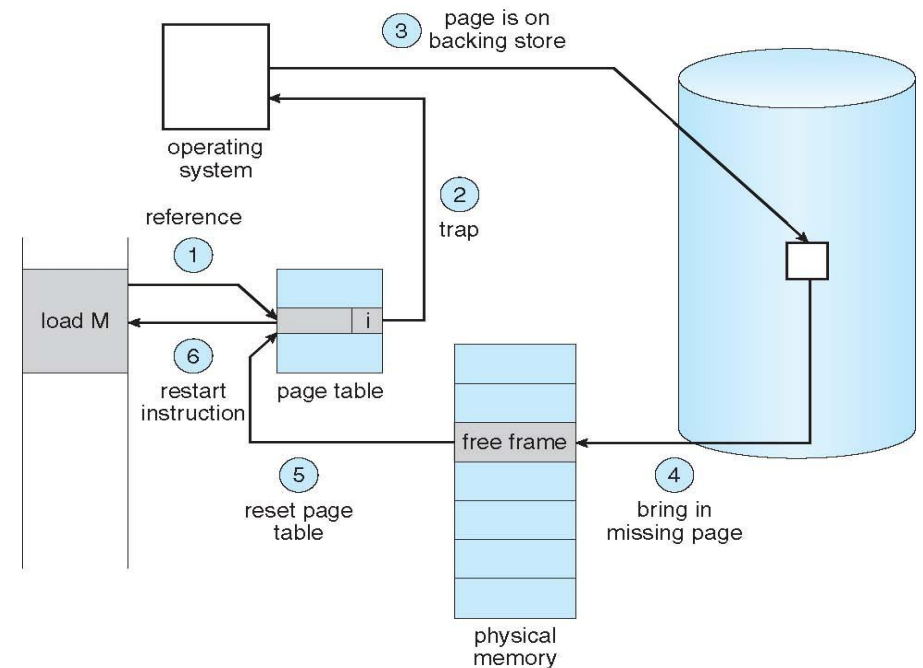
Steps in Handling a Page Fault



Performance of Demand Paging

Stages in Demand Paging

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
 1. Wait in a queue for this device until the read request is serviced
 2. Wait for the device seek and/or latency time
 3. Begin the transfer of the page to a free frame
6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction



Performance of Demand Paging

Page Fault Rate $0 \leq p \leq 1$

if $p = 0$ no page faults

if $p = 1$, every reference is a fault

Effective Access Time (EAT)

$$\begin{aligned} \text{EAT} = & (1 - p) \times \text{memory access} \\ & + p (\text{page fault overhead} \\ & \quad + \text{swap page out} \\ & \quad + \text{swap page in} \\ & \quad + \text{restart overhead}) \end{aligned}$$

Example :

Memory access time = 200 nanoseconds

Average page-fault service time = 8 milliseconds

$$\begin{aligned} \text{EAT} &= (1 - p) \times 200 + p (8 \text{ milliseconds}) \\ &= (1 - p) \times 200 + p \times 8,000,000 = 200 + p \times 7,999,800 \end{aligned}$$

If one access out of 1,000 causes a page fault, then $\text{EAT} = 8.2 \text{ microseconds}$.

This is a slowdown by a factor of 40!!

If want performance degradation < 10 percent

$$220 > 200 + 7,999,800 \times p$$

$$20 > 7,999,800 \times p$$

$$p < .0000025$$

< one page fault in every 400,000 memory accesses

What Happens if There is no Free Frame?

Page replacement is a virtual-memory technique used when a process generates a **page fault** but **no free physical-memory frame is available**.

The operating system must:

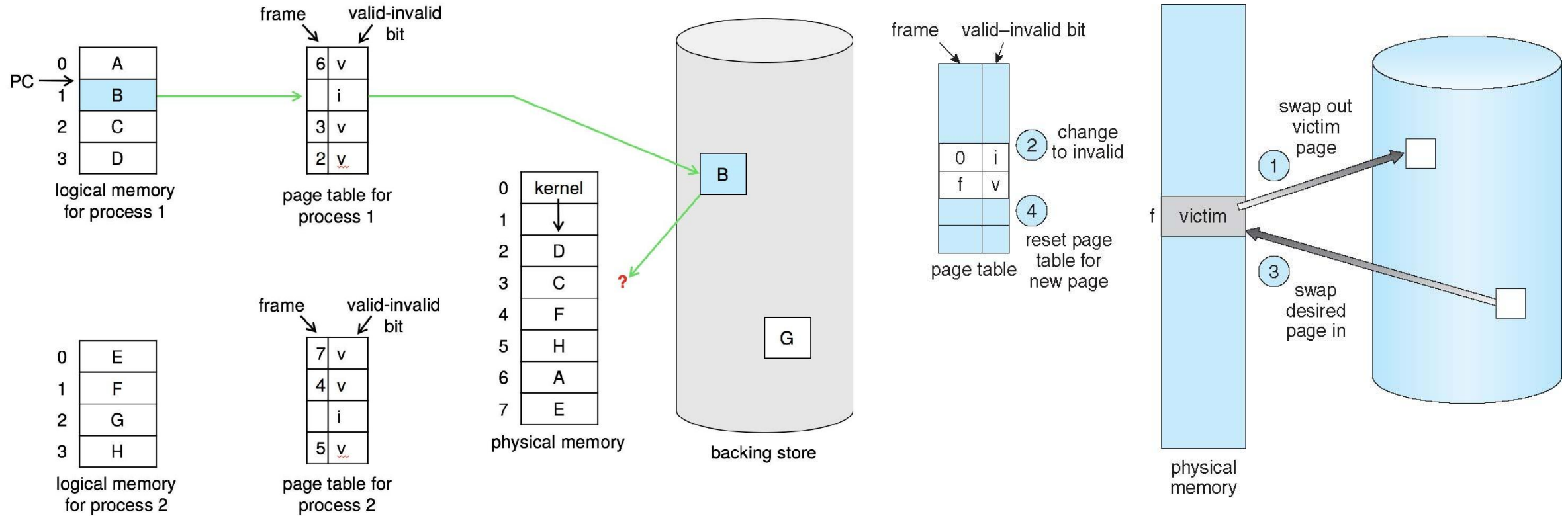
1. Choose an existing page in memory as a **victim page**.
2. Remove that page from its frame.
3. Load the required page into the freed frame.
4. Update the page table.
5. Restart the interrupted instruction.

The main objective of a page-replacement algorithm is to **minimize the number of page faults**.

Page Replacement Algorithms:

1. FIFO (First in First Out)
2. Optimal
3. LRU (Least recently Used)
4. Second-Chance
5. LFU (Least frequently Used)
6. MFU (Most Frequently Used)

General Page Replacement Procedure



What Happens if There is no Free Frame?

Frame-allocation algorithm determines

- How many frames to give each process
- Which frames to replace

Page-replacement algorithm

- Want lowest page-fault rate on both first access and re-access

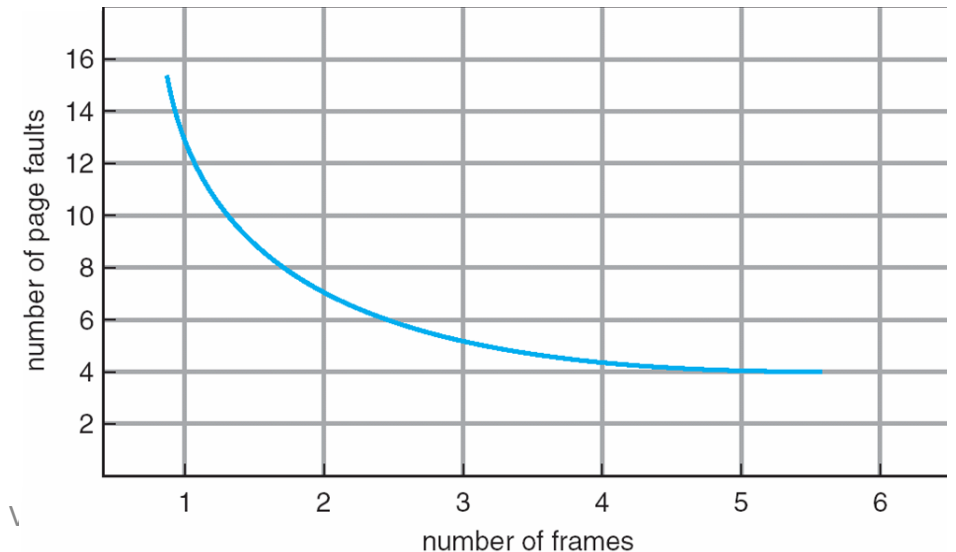
Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string

- String is just page numbers, not full addresses
- Repeated access to the same page does not cause a page fault
- Results depend on number of frames available

In all our examples, the **reference string** of referenced page numbers is

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

Graph of Page Faults Versus the Number of Frames



FIFO Page Replacement

- FIFO removes the page that has been in memory for the **longest time**.
- It behaves like a queue.
- Oldest page → removed first

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	2	4	4	4	0	0	0	7	7	7
	0	0	0	3	3	3	2	2	2	1	1	1	0	0
		1	1	1	0	0	0	3	3	3	2	2	2	1

15 page faults

page frames

Advantages

- Very simple.
- Easy to implement.
- Requires little bookkeeping.

Disadvantages

- FIFO does not consider how frequently or recently a page is being used.
- A heavily used page may be removed simply because it is old.

FIFO Page Replacement

An important problem associated with FIFO is **Belady's anomaly**.

Normally, we expect:

More frames $\rightarrow$ fewer page faults.

But with FIFO, sometimes:

Increasing the number of frames increases the number of page faults.

Example reference string:

1 2 3 4 1 2 5 1 2 3 4 5

For FIFO:

3 frames $\rightarrow$ 9 page faults

4 frames $\rightarrow$ 10 page faults

This unexpected behavior is called **Belady's anomaly**.

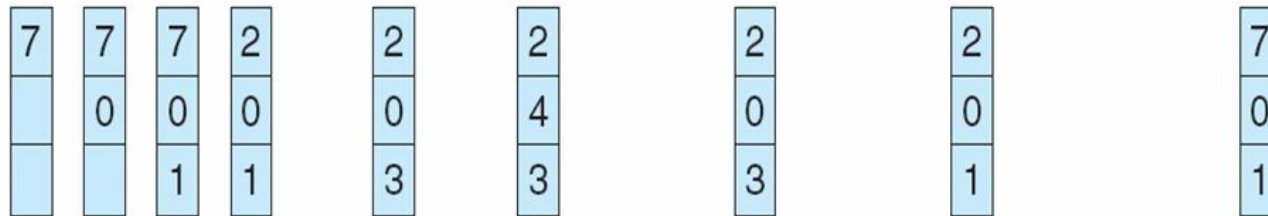
Optimal Page Replacement

Replace page that will not be used for longest period of time

9 is optimal for the example

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Advantage

Optimal produces the **minimum possible number of page faults** for a given reference string.

Major disadvantage

- The OS cannot normally know future memory references.
- Therefore, Optimal is mainly used as a:

benchmark to compare other page replacement algorithms.

Least Recently Used (LRU) Algorithm

- Replaces the page that has the **smallest reference count**, This is based on **locality of reference**..
- **Idea**: an actively used page should have a large reference count. If a page has a low count, it means it is rarely or barely used.
- A page might be used heavily during the initial phase of a process and accumulate a massive count, but then never be needed again. It stays in memory forever unless counters are periodically shifted right (decayed).

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

Least Recently Used (LRU) Algorithm

- The operating system assigns a reference counter to each page in memory.
- Each time a page is referenced (accessed by the CPU), its counter increases by one.
- When a page fault occurs and memory is full, the OS looks at these counters to decide which page to remove.
- Because they require extra overhead to maintain and do not closely approximate the optimal (OPT) page replacement strategy, they are rarely used in standard systems.

Types of Counting Algorithms

- **Least Frequently Used (LFU)**
 - Replaces the page that has the **smallest reference count**.
 - **Idea:** an actively used page should have a large reference count. If a page has a low count, it means it is rarely or barely used.
 - A page might be used heavily during the initial phase of a process and accumulate a massive count, but then never be needed again. It stays in memory forever unless counters are periodically shifted right (decayed).
- **Most Frequently Used (MFU)**
 - Replaces the page with the **highest reference count**.
 - **Idea:** Based on the argument that a page with the smallest count was probably just brought into memory and has yet to be used. Therefore, the one with the largest count has been used already and is no longer needed, or has run its course.
 - MFU is even less common than LFU because it poorly approximates ideal replacement behavior.

Counting based Algorithm - LFU

Frame size is 3

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
7	7	7	2	2	2	2	4	4	3	3	3	3	3	3
	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	1	2
F	F	F	F		F		F	F	F				F	F

Page Fault=10

Counting based Algorithm - MFU

Frame size is 4

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			5	5	5	5	5	5	5		5	5	5	5	5	5
	2	2	2			2	6	6	6	6	6	6		3	2	1	2	3	6
		3	3			3	3	2	1	2	3	7		7	7	7	7	7	7
			4			4	4	4	4	4	4	4		4	4	4	4	4	4

Page Fault = 17

Page Fault=17

Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high
 - Page fault to get page
 - Replace existing frame
 - But quickly need replaced frame back
 - This leads to:
 - Low CPU utilization
 - Operating system thinking that it needs to increase the degree of multiprogramming
 - Another process added to the system